

PayPal TX Ledger Guide (Codebase-Verified, Chronological, Ledger-Focused)

This guide is based on your current codebase structure and existing PayPal checkout flow. It is a blueprint + implementation templates document (not yet applied runtime code).

Date verified against codebase: 2026-02-24

Runtime checkpoint for false-success fulfillment handling: 2026-06-16

Implemented in the current code checkpoint:

- The Django fulfillment process endpoint is no longer treated as successful from HTTP 2xx alone.
- A response body with `success: false`, `processing_status: "failed"`, or `error_message` is normalized to a fulfillment business failure.
- False-success fulfillment failures end as `status = "fulfillment_failed"` with `lastErrorCode = "FULFILLMENT_PROVIDER_REJECTED"`.
- Local fulfillment payload validation failures end as `status = "fulfillment_blocked"` with `lastErrorCode = "FULFILLMENT_PAYLOAD_INVALID"`.
- The ledger stores the canonical fulfillment request payload and the raw/local failure response payload for both recovery states.
- Automatic capture/webhook resume paths should not move `fulfillment_failed` or `fulfillment_blocked` rows backward to `captured`; admin recovery owns those rows.

Not part of this checkpoint:

- `AdminNotificationOutbox` table/helper/email delivery is still the next admin tooling step.
- The admin dashboard revamp should consume these durable ledger states and payloads; it should not redesign the payment state machine.

Table of Contents

1. Scope and Ground Rules
2. Current Flow (Verified from Code)
3. What We Are Changing (and Not Changing)
4. Final Target Flow (End-to-End)
5. Implementation Order (Chronological)
6. Environment Variables
7. Prisma Schema (PayPal Ledger) - Complete and Typed
8. Prisma Commands (No Separate Prisma Config File)
9. Prisma Client Wrapper
10. Status Constants and Vocabulary (string-based, enum deferred)
11. Ledger Types (JSON payload typing strategy)
12. Checkout Intent Store (client)
13. PayPal Intent Endpoint (server)
14. PayPalCheckoutChildren Integration (replace createOrderAction path)

15. Authorize Route Update (ledger write)
 16. Capture Route Update (ledger write)
 17. Approve Hook Update (`usePayPalTXApproveCallback`)
 18. Order Status Endpoint
 19. Polling + UI Mapping (Processing Modal / Confirmation Page)
 20. Webhook Route Upgrade (idempotent + post-processing trigger)
 21. Server Post-Processing Runner (ledger-driven)
 22. Fulfillment Provider Adapter (Merchize isolated)
 23. Admin Auth (header secret)
 24. Admin Ledger UI + Retry Actions
 25. Sandbox/Dev Fallback Trigger
 26. Rollout and Migration Order (safe deployment)
 27. Package Scripts (no extra Prisma config file)
 28. Testing Matrix (local/sandbox/live)
 29. Decision Defaults
 30. Optional Overrides (only if rejecting defaults)
 31. Future: Real-Time Notifications at Scale (SSE + Pub/Sub)
-

1) Scope and Ground Rules

Goal

Move post-payment processing (receipt generation/upload, payment save, fulfillment push) out of the browser and into a durable server-side ledger + webhook flow.

Non-goals

- Rewriting your entire checkout UI.
- Replacing your existing PayPal order creation route.
- Replacing your OTP flow.

Ground rules for this guide

- Use your existing route naming convention (`/next-api/...`).
 - Keep Merchize SQLite setup untouched.
 - Do **not** require a separate Prisma config file for the PayPal ledger schema.
 - Keep status fields as strings for now (Prisma enum deferred).
 - Isolate fulfillment-provider details behind a ledger runner adapter boundary.
-

2) Current Flow (Verified from Code)

2.1 Checkout + OTP

- Checkout info form lives in:
 - `src/components/UI/Shop/Checkout/UserCheckoutSummary/BasicCheckoutInfo.tsx`
- Django order-intent OTP send/verify flow lives in:
 - `src/lib/hooks/shopHooks/checkout/useDjangoOrderIntentEmailVerification.ts`
 - `src/lib/hooks/shopHooks/checkout/djangoOrderIntentMutationHooks.ts`
- The browser does not sign Django order-intent requests directly. The client hook calls local Next routes:
 - `/next-api/shop/checkout/django-order-intent`
 - `/next-api/shop/checkout/django-order-intent/verify`
 - `/next-api/shop/checkout/django-order-intent/resend-otp`
- Those Next routes forward to Django through:
 - `src/lib/shop/checkout/djangoOrderIntent/djangoOrderIntentServer.ts`
 - `src/lib/hooks/shopHooks/checkout/helpers/generateSignatureHeaders.ts`
- HMAC generation stays server-side. Do not reintroduce a `NEXT_PUBLIC_*` OTP signing secret into a client hook.
- Verified Django order-intent identifiers are stored in:
 - `src/stores/shop_stores/checkoutStore/djangoOrderIntentStore.ts`

2.2 PayPal order creation

- `src/components/UI/Shop/Checkout/Paypal/PayPalCheckoutChildren.tsx` calls `createOrderAction(...)`.
- `src/actions/shop/paypal/createOrderAction.ts` decrypts payload and calls:
 - `/next-api/paypal/orders/create-order`
- `src/app/api/paypal/orders/create-order/route.ts` validates pricing via `getOrderFinalDetails(...)` and creates the PayPal order.

2.3 Approve, authorize, capture

- `src/lib/hooks/shopHooks/checkout/usePayPalTXApproveCallback.ts` calls:
 - `/next-api/paypal/orders/authorize`
 - `/next-api/paypal/orders/capture`
- Routes:
 - `src/app/api/paypal/orders/authorize/route.ts`
 - `src/app/api/paypal/orders/capture/route.ts`

2.4 Post-processing (currently client-oriented but partially disabled)

The intended client path is:

- `savePaymentReceiptToCloud`
- `savePaymentDataToBackend`
- `sendMerchizeOrderDetailsToBackend`

Current state:

- `usePayPalTXApproveCallback.ts` starts processing UI but post-processing call is commented out.
- This can leave the modal active without progress completion.

2.5 Webhook

- `src/app/api/paypal/webhooks/ledger-transaction-events/route.ts` already includes:
 - raw-body parsing pattern (`req.text()`)
 - optional signature verification
 - Business processing in webhook is still placeholder comments.
-

3) What We Are Changing (and Not Changing)

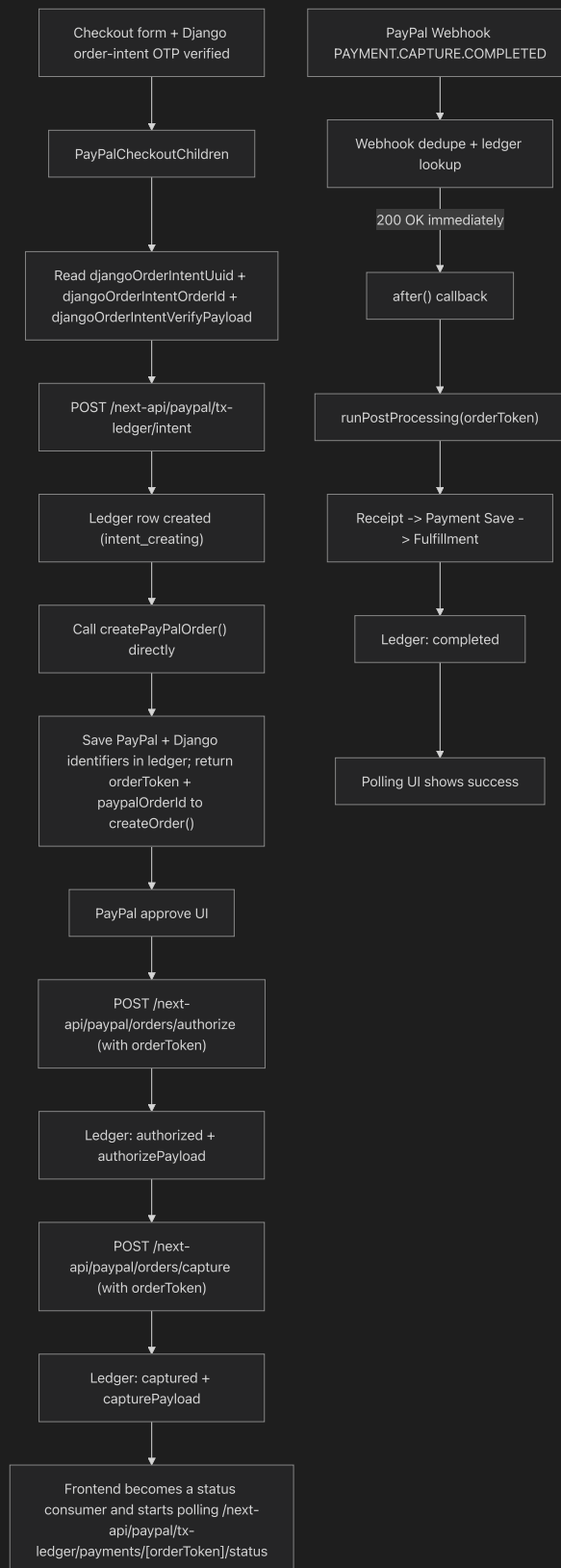
We are changing

- Add PayPal ledger database tables (Neon/Postgres via Prisma).
- Extract create-order business logic into `src/lib/paypal/createPayPalOrder.ts` (shared function).
- Refactor `create-order` route to be a thin wrapper around the shared function.
- Add a new `/next-api/paypal/tx-ledger/intent` route that calls the shared function directly (no self-referential HTTP fetch).
- Write ledger state during authorize/capture.
- Make webhook idempotent and trigger server post-processing.
- Add `/next-api/paypal/tx-ledger/payments/[orderToken]/status` polling route.
- Add admin ledger retry UI/actions.

We are not changing

- Existing `/next-api/paypal/orders/create-order` route behavior (same inputs/outputs, logic extracted to shared function).
 - Existing `/next-api/paypal/orders/authorize` and `/capture` endpoints as the ownership point for authorize/capture.
 - OTP verification backend contract.
 - Merchize SQLite catalog DB and its Prisma config.
-

4) Final Target Flow (End-to-End)



5) Implementation Order (Chronological)

Implement in this order to avoid half-migrated behavior:

1. Add ledger schema + generate client.

2. Add status constants/types.
3. Add ledger client wrapper.
4. Add intent store (client).
5. Move Django order-intent HMAC signing behind the local Next proxy routes. The browser must not sign `/orders/intent`, `/verify`, or `/resend-otp` directly.
6. Extract create-order logic into `src/lib/paypal/createPayPalOrder.ts` and refactor `create-order` route to use it.
7. Add `/next-api/paypal/tx-ledger/intent` (calls `createPayPalOrder` directly, no self-referential fetch).
8. Update `PayPalCheckoutChildren` to use intent route.
9. Update authorize route to persist ledger.
10. Update capture route to persist ledger.
11. Update approve hook to pass `orderToken`, remove client post-processing, and redirect into the confirmation/status flow.
12. Add `/next-api/paypal/tx-ledger/payments/[orderToken]/status`.
13. Add polling + UI mapping.
14. Upgrade webhook (idempotency + ledger updates + post-processing trigger).
15. Add server post-processing runner + fulfillment adapter.
16. Add admin auth + admin UI/actions.
17. Add dev fallback trigger endpoint.
18. Retire client-side post-processing from checkout runtime.

Important transition note:

- After Step 11, the old browser-owned completion path is intentionally gone.
 - Step 12 and Step 13 restore the frontend read path.
 - Step 14 and Step 15 restore durable backend completion.
 - So if you adopt the thin approve hook early, do not expect receipt upload / payment save / fulfillment to run again until Steps 13 and 14 are implemented.
-

6) Environment Variables

TEXT

```
# Existing (already in project)
DATABASE_URL="file:./dev.db"
PAYPAL_WEBHOOK_VERIFY="true"
PAYPAL_ENV="sandbox"
NEXT_PUBLIC_PAYPAL_ENV="sandbox"

# Preferred PayPal browser vars
NEXT_PUBLIC_PAYPAL_CLIENT_ID_SANDBOX="..."
NEXT_PUBLIC_PAYPAL_CLIENT_ID_LIVE="..."

# Preferred PayPal server vars
PAYPAL_CLIENT_ID_SANDBOX="..."
PAYPAL_CLIENT_ID_LIVE="..."
PAYPAL_CLIENT_SECRET_SANDBOX="..."
PAYPAL_CLIENT_SECRET_LIVE="..."
PAYPAL_WEBHOOK_ID_SANDBOX="..."
PAYPAL_WEBHOOK_ID_LIVE="..."

# Backward-compatible fallback vars (supported, but no longer preferred)
NEXT_PUBLIC_PAYPAL_CLIENT_ID="..."
PAYPAL_CLIENT_ID="..."
PAYPAL_CLIENT_SECRET="..."
PAYPAL_WEBHOOK_ID="..."

# Browser-reachable Django backend base URL
NEXT_PUBLIC_DJANGO_API_BASE_URL="..."

# Optional server-to-server Django base URL for same-network production calls
DJANGO_INTERNAL_BASE_URL="http://django:8000/api/v1"

# Django order-intent HMAC signing secret (server-only)
SHOP_CHECKOUT_OTP_VERIFICATION_API_KEY="..."

# New (ledger)
PAYPAL_TX_LEDGER_NEON_DB_STRING="postgresql://USER:PASSWORD@HOST/DB?sslmode=require"
ADMIN_SECRET_KEY="long-random-secret"
SHOP_SERVER_ACTIONS_CRYPT0_SECRET="long-random-secret"

# Admin recovery notifications
ORDER_RECOVERY_ADMIN_EMAILS="ops@example.com,support@example.com"
ORDER_RECOVERY_NOTIFICATION_FROM_EMAIL="support@example.com"
```

Notes:

- Browser PayPal config is resolved from:
 - `src/lib/paypal/publicPayPalConfig.ts`
- Server PayPal config is resolved from:

- `src/lib/paypal/serverPayPalConfig.ts`
- `NEXT_PUBLIC_PAYPAL_CLIENT_ID_*` values are public by design. They are not secrets.
- `PAYPAL_CLIENT_SECRET_*` and `PAYPAL_WEBHOOK_ID_*` stay server-only.
- The current code supports both the new split variables and the old single-name fallbacks during migration.
- For clean deployments, keep `PAYPAL_ENV` and `NEXT_PUBLIC_PAYPAL_ENV` aligned.
- Browser code uses `NEXT_PUBLIC_DJANGO_API_BASE_URL`.
- Server-only Django calls may use `DJANGO_INTERNAL_BASE_URL` when Next and Django share a private Docker network; otherwise they fall back to the public Django API base.
- Merchize SQLite continues to use your existing `prisma.config.ts`.
- PayPal ledger schema also gets its datasource URL from `prisma.config.ts` in Prisma 7.
- The root `prisma.config.ts` should switch datasource URLs based on the `--schema` target.
- Django order-intent HMAC signing is server-only. The browser should call the local Next proxy routes and should never receive the signing secret.
- Do **not** reuse `SHOP_CHECKOUT_OTP_VERIFICATION_API_KEY` for webhook/server post-processing. Keep a separate server-only secret for those actions.
- `ORDER_RECOVERY_ADMIN_EMAILS` receives internal alerts when a paid order needs admin recovery. Email delivery must be driven from a durable outbox row, not from an untracked fire-and-forget side effect.
- This guide intentionally avoids requiring a second Prisma config file for the ledger.

7) Prisma Schema (PayPal Ledger) - Complete and Typed

File:

- `prisma/shop/paypal/paypalTXLedger.schema.prisma`

This schema is ledger-focused and keeps fulfillment-provider specifics isolated from the schema itself.


```

datasource db {
  provider = "postgresql"
  // URL is set in prisma.config.ts (Prisma 7 style)
}

generator paypalTxLedger {
  provider = "prisma-client"
  output   = "../../src/lib/prisma/shop/paypal/txLedger/generated/paypalTxLedger"
}

model PaypalIntent {
  id                      String      @id @default(cuid())
  orderToken              String      @unique
  paypalOrderId           String?     @unique
  paypalAuthorizationId   String?
  djangoOrderIntentUuid   String?
  djangoOrderIntentOrderId String?
  customerName            String
  customerEmail           String
  userId                  String?
  countryIso2             String?
  countryIso3             String?
  initialCurrency         String?
  cartSnapshot            Json
  shippingSnapshot        Json
  // Raw payloads needed to reuse your existing server actions
  authorizePayload        Json?
  capturePayload          Json?
  djangoOrderIntentPayload Json?
  djangoOrderIntentVerifyPayload Json?
  djangoPaymentSaveResponsePayload Json?
  merchizeFulfillmentRequestPayload Json?
  merchizeFulfillmentResponsePayload Json?
  merchizeFulfillmentProcessingId String?
  merchizeProviderOrderId String?
  merchizeProviderOrderCode String?
  fulfillmentAddressOverride Json?
  fulfillmentAddressOverrideReason String?
  fulfillmentAddressOverriddenBy String?
  fulfillmentAddressOverriddenAt DateTime?
  djangoPaymentSaveCustomId String?
  receiptLink             String?
  receiptFile             String?
  status                  String
  lastEventType           String?
  lastErrorCode           String?
  lastErrorMessage        String?
  retryCount              Int         @default(0)
  postProcessingLockId    String?
  postProcessingLockedAt  DateTime?
  postProcessingLockExpiresAt DateTime?
  processingCompletedAt   DateTime?

```

```

        createdAt      DateTime @default(now())
        updatedAt      DateTime @updatedAt

        @@index([customerEmail])
        @@index([djangoOrderIntentUuid])
        @@index([djangoOrderIntentOrderId])
        @@index([djangoPaymentSaveCustomId])
        @@index([status])
        @@index([createdAt])
        @@index([postProcessingLockExpiresAt])
    }

model PaypalWebhookEvent {
    id          String   @id @default(cuid())
    eventId     String   @unique
    eventType   String
    paypalOrderId String?
    payload     Json
    processingStatus String @default("received")
    attemptCount Int      @default(0)
    lastAttemptAt DateTime?
    processedAt DateTime?
    lastErrorMessage String?
    createdAt   DateTime @default(now())

    @@index([paypalOrderId])
    @@index([eventType])
    @@index([processingStatus])
    @@index([processedAt])
    @@index([createdAt])
}

model AdminNotificationOutbox {
    id          String   @id @default(cuid())
    orderToken   String?
    paypalOrderId String?
    type        String
    stage       String?
    errorCode    String?
    severity     String   @default("warning")
    status      String   @default("pending")
    dedupeKey    String   @unique
    recipient   String?
    payload     Json
    attemptCount Int      @default(0)
    lastAttemptAt DateTime?
    sentAt      DateTime?
    lastErrorMessage String?
    createdAt   DateTime @default(now())
    updatedAt   DateTime @updatedAt

    @@index([orderToken])
    @@index([paypalOrderId])
    @@index([type])

```

```

    @@index([stage])
    @@index([errorCode])
    @@index([severity])
    @@index([status])
    @@index([createdAt])
  }

```

Admin notification intent:

- This table is the durable email queue for configured admin recipients.
- When a valid operational event occurs, create an outbox row for each configured recipient in `ORDER_RECOVERY_ADMIN_EMAILS`.
- The sender then delivers email from the outbox row and updates `status`, `attemptCount`, `sentAt`, and `lastErrorMessage`.
- The ledger/recovery state must not depend on email delivery succeeding. Email delivery is an alerting side effect with its own retry state.

Why no Prisma relation between `PaypalIntent` and `PaypalWebhookEvent`?

- PayPal webhook events may arrive before a ledger row is fully correlated.
- Event dedupe only needs `eventId`.
- A plain indexed `paypalOrderId` field is simpler and avoids relational coupling at this stage. A formal FK can be added when the schema stabilizes.
- Admin notifications use an outbox table with searchable transaction context (`orderToken`, `paypalOrderId`, `stage`, `errorCode`). Do not make email delivery part of the payment transaction; persist the outbox row first, then send from the outbox.

Why the extra tracking fields matter:

- `PaypalWebhookEvent.processedAt` prevents false "already handled" acks when a prior attempt failed.
- `PaypalIntent.postProcessingLock*` gives `runPostProcessing(...)` a DB-backed lease so only one worker can own side effects at a time.
- `AdminNotificationOutbox.dedupeKey` prevents repeated webhook/retry attempts from spamming admins for the same unresolved paid-order condition.

8) Prisma Commands (No Separate Prisma Config File)

Because you do **not** want a separate Prisma config file for the ledger:

- Keep `prisma.config.ts` for Merchize SQLite.
- Extend the root `prisma.config.ts` so it selects the correct datasource URL based on `--schema`.
- Require `--schema` explicitly for every Prisma command. Do not let the config guess.
- For PayPal ledger, run Prisma commands explicitly with `--schema`.

Generate PayPal ledger client

BASH

```
yarn prisma generate --schema prisma/shop/paypal/paypalTXLedger.schema.prisma
```

Create migration (development)

BASH

```
yarn prisma migrate dev \  
  --schema prisma/shop/paypal/paypalTXLedger.schema.prisma \  
  --name init_paypal_tx_ledger
```

Deploy migrations (production)

BASH

```
yarn prisma migrate deploy --schema prisma/shop/paypal/paypalTXLedger.schema.prisma
```

Inspect / validate

BASH

```
yarn prisma validate --schema prisma/shop/paypal/paypalTXLedger.schema.prisma  
yarn prisma format --schema prisma/shop/paypal/paypalTXLedger.schema.prisma
```

This avoids conflicts with:

- `prisma.config.ts` (Merchize / SQLite)

9) Prisma Client Wrapper

File:

- `src/lib/prisma/shop/paypal/paypalTxLedger.ts`

TS

```
import { PrismaClient } from '@lib/prisma/shop/paypal/txLedger/generated/paypalTxLedger/client';

declare global {
  // eslint-disable-next-line no-var
  var __paypalTxLedger__: PrismaClient | undefined;
}

export const paypalTxLedger = global.__paypalTxLedger__ ?? new PrismaClient();

if (process.env.NODE_ENV !== 'production') {
  global.__paypalTxLedger__ = paypalTxLedger;
}
```

Why this wrapper:

- Prevents connection churn in dev/hot reload.
- Gives one import path used by routes/hooks/server functions.

10) Status Constants and Vocabulary (string-based, enum deferred)

Decision default is string statuses (centralized constants), with Prisma enum conversion deferred.

File:

- `src/lib/paypal/txLedger/status.ts`

TS

```
export const PAYPAL_LEDGER_STATUS = {
  INTENT_CREATING: 'intent_creating',
  INTENT_CREATED: 'intent_created',
  AUTHORIZED: 'authorized',
  CAPTURED: 'captured',
  RECEIPT_UPLOADED: 'receipt_uploaded',
  PAYMENT_SAVED: 'payment_saved',
  COMPLETED: 'completed',
  FULFILLMENT_BLOCKED: 'fulfillment_blocked',
  FULFILLMENT_FAILED: 'fulfillment_failed',
  PENDING: 'pending',
  REFUNDED: 'refunded',
  ERROR: 'error',
} as const;

export type PayPalLedgerStatus =
  (typeof PAYPAL_LEDGER_STATUS)[keyof typeof PAYPAL_LEDGER_STATUS];
```

Why this matters:

- Prevents typos while keeping Prisma schema flexible.
- `fulfillment_blocked` means the local payload is not fit to send or retry without admin repair, such as a missing address field or un-normalizable country.
- `fulfillment_failed` means the fulfillment provider/Django process endpoint returned a business failure after transport succeeded.

Error code defaults:

TS

```
export const PAYPAL_LEDGER_ERROR_CODE = {
  FULFILLMENT_PAYLOAD_INVALID: 'FULFILLMENT_PAYLOAD_INVALID',
  FULFILLMENT_PROVIDER_REJECTED: 'FULFILLMENT_PROVIDER_REJECTED',
  POST_PROCESSING_FAILED: 'POST_PROCESSING_FAILED',
} as const;
```

Do not collapse fulfillment validation failures and provider business rejections into generic

`POST_PROCESSING_FAILED`; admins need to know whether to repair local data or investigate the downstream provider.

11) Ledger Types (JSON payload typing strategy)

Store raw JSON in DB, but type it at read boundaries.

File:

- `src/lib/paypal/txLedger/types.ts`

TS

```
import type { OrderResponseBody } from '@paypal/paypal-js';
import type { OrdersCapture } from '@paypal/paypal-server-sdk';

export type LedgerAuthorizePayload = OrderResponseBody;
export type LedgerCapturePayload = OrdersCapture;

export type LedgerCartSnapshotItem = {
  variantId: string;
  quantity: number;
  title?: string;
  slug?: string;
  itemDetail?: unknown;
};
```

Policy:

- Store raw responses exactly as returned by existing routes.
 - Normalize only when needed by post-processing or UI.
-

12) Checkout Intent Store (client)

File:

- `src/stores/shop_stores/checkoutStore/paypalIntentStore.ts`

TS

```
'use client';
import { create } from 'zustand';

type PayPalIntentState = {
  orderToken?: string;
  setIntent: (payload: { orderToken: string }) => void;
  clearIntent: () => void;
};

export const usePayPalIntentStore = create<PayPalIntentState>((set) => ({
  orderToken: undefined,
  setIntent: ({ orderToken }) => set({ orderToken }),
  clearIntent: () => set({ orderToken: undefined }),
}));
```

This is intentionally small and separate from your existing checkout store.

Design note:

- Keep only `orderToken` in browser app state.
- Keep `paypalOrderId` as a server-side ledger correlation value.
- The browser still briefly receives `paypalOrderId` because PayPal's SDK requires `createOrder()` to return it, but your own client state should not retain it after that callback returns.

13) PayPal Intent Endpoint (server)

13a) Extract create-order logic into a shared function

The existing `create-order` route handler contains the full order-creation business logic inline. Before adding the intent endpoint, extract that logic into a directly-importable function so the intent route can call it without an HTTP round-trip to itself.

Why this matters:

- A self-referential `fetch()` (route calling its own server) can deadlock under load when all Node.js workers are occupied waiting on themselves.
- Direct function calls are faster, testable, and do not depend on `host` header availability.

New file:

- `src/lib/paypal/createPayPalOrder.ts`

```

import {
  OrdersController,
  CheckoutPaymentIntent,
  OrderRequest,
  ItemCategory,
  Item,
} from '@paypal/paypal-server-sdk';
import { paypalClient } from '@lib/paymentClients/paypalClient';
import { getOrderFinalDetails } from '@actions/shop/checkout/getOrderFinalDetails';
import { PAYPAL_CURRENCY_CODES } from '@datasets/shop_general/paypal_currency_specifics';
import { removeOrKeepDecimalPrecision } from '@actions/merchize/getMerchizeTotalWithShipping';
import { format } from 'date-fns';

const orders = new OrdersController(paypalClient);

const shippingPreferenceForPaymentContext = {
  experienceContext: {
    shippingPreference: 'SET_PROVIDED_ADDRESS',
    brandName: 'Codex Christi',
  },
};

export type CreatePayPalOrderArgs = {
  orderToken: string;
  cart: Array<{ title: string; itemDetail: any; quantity: number; [k: string]: any }>;
  customer: { name: string; email: string };
  country: string;
  country_iso_3?: string;
  initialCurrency?: string;
  delivery_address: {
    shipping_address_line_1: string;
    shipping_address_line_2?: string;
    shipping_state?: string;
    shipping_city?: string;
    zip_code?: string;
    [k: string]: any;
  };
};

/**
 * Core order-creation logic extracted from the create-order route.
 * Returns the raw PayPal order response body (includes `.id`).
 * Throws on validation or PayPal API failure.
 */
export async function createPayPalOrder(args: CreatePayPalOrderArgs) {
  const { orderToken, cart, customer, country, country_iso_3, initialCurrency, delivery_address } =
    args;

  if (!orderToken) throw new Error('Missing order token');
  if (!cart || cart.length === 0) throw new Error('Missing cart');
  if (!customer) throw new Error('Missing customer');

```



```

const paypalSupportsCurrency = PAYPAL_CURRENCY_CODES.includes(
  initialCurrency as (typeof PAYPAL_CURRENCY_CODES)[number],
);

const orderDetailsFromServer = await getOrderFinalDetails(
  cart,
  country_iso_3 ?? 'USA',
  'merchize',
);

const { finalPricesWithShippingFee } = orderDetailsFromServer || {};
const { currency, retailPriceTotalNum, shippingPriceNum, multiplier } =
  finalPricesWithShippingFee || {};

const currencyCode = currency && paypalSupportsCurrency ? currency : 'USD';
const getAdjAmount = (num: number) => (paypalSupportsCurrency ? num : num / (multiplier ?? 1));
const adjShipping = getAdjAmount(shippingPriceNum!);

const cartItemsForPaypalBodyContext = cart.map(
  async ({ title, itemDetail, quantity }) =>
    ({
      name: title,
      unitAmount: {
        currencyCode: currency,
        value: String(
          await removeOrKeepDecimalPrecision(
            currency!,
            getAdjAmount(itemDetail.retail_price * (multiplier ?? 1)),
          ),
        ),
      },
      quantity: String(quantity),
      description: title,
      sku: itemDetail.sku_seller,
      url: `https://codexchristi.shop/product/${itemDetail.product}`,
      category: ItemCategory.PhysicalGoods,
    }) as Item,
);

if (!retailPriceTotalNum || !shippingPriceNum) {
  throw new Error('Invalid Price!! Aborting...');
}

const resolvedCartItems = await Promise.all(cartItemsForPaypalBodyContext);
const itemsContextTotal = await removeOrKeepDecimalPrecision(
  currencyCode,
  resolvedCartItems.reduce(
    (accum, itm) => accum + Number(itm.unitAmount.value) * Number(itm.quantity),
    0,
  ),
);

const payload = {
  body: {
    intent: 'AUTHORIZE' as CheckoutPaymentIntent,

```

```

purchaseUnits: [
  {
    description: `Codex Christi Shop Order for ${customer.name} on ${format(
      new Date(Date.now()),
      "EEEE d 'of' MMMM yyyy hh:mm a",
    )}`,
    amount: {
      currencyCode,
      value: String(
        await removeOrKeepDecimalPrecision(currencyCode, itemsContextTotal + adjShipping),
      ),
      breakdown: {
        itemTotal: { currencyCode, value: String(itemsContextTotal) },
        shipping: {
          currencyCode,
          value: String(await removeOrKeepDecimalPrecision(currencyCode, adjShipping)),
        },
      },
    },
    shipping: {
      name: { fullName: customer.name },
      address: {
        addressLine1: delivery_address.shipping_address_line_1,
        addressLine2: delivery_address.shipping_address_line_2,
        adminArea1: delivery_address.shipping_state,
        adminArea2: delivery_address.shipping_city,
        postalCode: delivery_address.zip_code,
        countryCode: country,
      },
      emailAddress: customer.email,
    },
    customId: orderToken,
    items: paypalSupportsCurrency ? resolvedCartItems : undefined,
  },
],
payer: {
  name: { givenName: customer.name },
  emailAddress: customer.email,
},
paymentSource: {
  paypal: shippingPreferenceForPaymentContext,
  card: {
    experienceContext: shippingPreferenceForPaymentContext.experienceContext,
    attributes: { verification: { method: 'SCA_WHEN_REQUIRED' } },
  },
  venmo: shippingPreferenceForPaymentContext,
},
} as OrderRequest,
prefer: 'return=representation',
};

const { body: resBody } = await orders.createOrder(payload);
return resBody;
}

```

Then update the existing create-order route to use it:

Updated file:

- `src/app/api/paypal/orders/create-order/route.ts`

TS

```
import { NextResponse } from 'next/server';
import { createPayPalOrder } from '@lib/paypal/createPayPalOrder';

export async function POST(req: Request) {
  try {
    const body = await req.json();
    const resBody = await createPayPalOrder(body);
    return NextResponse.json(resBody);
  } catch (err: unknown) {
    console.error('Create Order Error', err);
    return NextResponse.json({ error: err instanceof Error ? err.message : String(err) }, { status: 500 });
  }
}
```

13b) Intent endpoint (calls `createPayPalOrder` directly)

New file:

- `src/app/api/paypal/tx-ledger/intent/route.ts`

Purpose:

- Create ledger row first.
- Call `createPayPalOrder(...)` directly (no self-referential HTTP fetch).
- Return `orderToken + paypalOrderId`.

Important ownership note:

- Store only `orderToken` in browser state.
- Return `paypalOrderId` only so `createOrder()` can hand it to the PayPal SDK.
- PayPal `purchase_units[].custom_id` should be set to `orderToken`. It is a local ledger correlation key, not the Django payment-save `custom_id`.
- After capture succeeds, the client should act only as a status consumer. Durable completion belongs to webhook + server runner.

```

import { NextResponse } from 'next/server';
import { randomUUID } from 'crypto';
import { paypalTxLedger } from '@lib/prisma/shop/paypal/paypalTxLedger';
import { PAYPAL_LEDGER_STATUS } from '@lib/paypal/txLedger/status';
import { createPayPalOrder } from '@lib/paypal/createPayPalOrder';

export async function POST(req: Request) {
  try {
    const body = await req.json();
    const {
      cart,
      customer,
      country,
      country_iso_3,
      initialCurrency,
      delivery_address,
      userId,
      djangoOrderIntentUuid,
      djangoOrderIntentOrderId,
      djangoOrderIntentPayload,
      djangoOrderIntentVerifyPayload,
    } = body;

    if (!Array.isArray(cart) || cart.length === 0) {
      return NextResponse.json({ error: 'Cart required' }, { status: 400 });
    }
    if (!customer?.name || !customer?.email) {
      return NextResponse.json({ error: 'Customer required' }, { status: 400 });
    }
    if (!delivery_address) {
      return NextResponse.json({ error: 'Delivery address required' }, { status: 400 });
    }

    const orderToken = randomUUID();

    await paypalTxLedger.paypalIntent.create({
      data: {
        orderToken,
        status: PAYPAL_LEDGER_STATUS.INTENT_CREATING,
        customerName: customer.name,
        customerEmail: customer.email,
        userId: userId ?? null,
        djangoOrderIntentUuid: djangoOrderIntentUuid ?? null,
        djangoOrderIntentOrderId: djangoOrderIntentOrderId ?? null,
        djangoOrderIntentPayload: djangoOrderIntentPayload ?? null,
        djangoOrderIntentVerifyPayload: djangoOrderIntentVerifyPayload ?? null,
        countryIso2: country ?? null,
        countryIso3: country_iso_3 ?? null,
        initialCurrency: initialCurrency ?? null,
        cartSnapshot: cart,
        shippingSnapshot: delivery_address,
      },
    },

```

```

});

let paypalOrder: any;
try {
  paypalOrder = await createPayPalOrder({
    orderToken,
    cart,
    customer,
    country,
    country_iso_3,
    initialCurrency,
    delivery_address,
  });
} catch (createErr) {
  await paypalTxLedger.paypalIntent.update({
    where: { orderToken },
    data: {
      status: PAYPAL_LEDGER_STATUS.ERROR,
      lastErrorCode: 'CREATE_ORDER_FAILED',
      lastErrorMessage: createErr instanceof Error ? createErr.message : String(createErr),
    },
  });
  return NextResponse.json({ error: 'Create order failed' }, { status: 500 });
}

if (!paypalOrder?.id) {
  await paypalTxLedger.paypalIntent.update({
    where: { orderToken },
    data: {
      status: PAYPAL_LEDGER_STATUS.ERROR,
      lastErrorCode: 'MISSING_PAYPAL_ORDER_ID',
      lastErrorMessage: 'createPayPalOrder returned no id',
    },
  });
  return NextResponse.json({ error: 'Missing PayPal order ID' }, { status: 500 });
}

await paypalTxLedger.paypalIntent.update({
  where: { orderToken },
  data: {
    paypalOrderId: paypalOrder.id,
    status: PAYPAL_LEDGER_STATUS.INTENT_CREATED,
  },
});

return NextResponse.json({
  orderToken,
  paypalOrderId: paypalOrder.id,
});
} catch (err) {
  return NextResponse.json({ error: 'Intent failed' }, { status: 500 });
}
}

```

14) PayPalCheckoutChildren Integration (replace createOrderAction path)

Current file:

- `src/components/UI/Shop/Checkout/Paypal/PayPalCheckoutChildren.tsx`

Replace the `createOrder` callback to call `/next-api/paypal/tx-ledger/intent`.

Important behavior:

- The intent route returns both `orderToken` and `paypalOrderId`.
- Store only `orderToken` in `paypalIntentStore`.
- Return `paypalOrderId` directly to the PayPal SDK.
- Do not persist `paypalOrderId` in your own client app state after `createOrder()` returns.

Required payload sources (codebase-verified)

- `cart` from `useCartStore`
- `customer` + `delivery_address` from `useShopCheckoutStore`
- `country_iso2`, `country_iso3`, `currency` from `ServerOrderDetailsContext`
- `djangoOrderIntentUuid`, `djangoOrderIntentOrderId`, and `djangoOrderIntentVerifyPayload` from `djangoOrderIntentStore`

Template (drop-in replacement shape)

TS

```
import { useDjangoOrderIntentStore } from '@stores/shop_stores/checkoutStore/djangoOrderIntentStore';
import { usePayPalIntentStore } from '@stores/shop_stores/checkoutStore/paypalIntentStore';

// inside component
const {
  djangoOrderIntentUuid,
  djangoOrderIntentOrderId,
  djangoOrderIntentPayload,
  djangoOrderIntentVerifyPayload,
} = useDjangoOrderIntentStore();
const setIntent = usePayPalIntentStore((s) => s.setIntent);

const createOrder = useCallback(async (): Promise<string> => {
  const payload = {
    cart,
    customer: {
      name: `${first_name} ${last_name}`,
      email: email ?? 'john@example.com',
    },
    country: country_iso2 ?? 'US',
    country_iso_3: country_iso3 ?? 'USA',
    initialCurrency: currency ?? 'USD',
    delivery_address,
    djangoOrderIntentUuid,
    djangoOrderIntentOrderId,
    djangoOrderIntentPayload,
    djangoOrderIntentVerifyPayload,
  };

  const res = await fetch('/next-api/paypal/tx-ledger/intent', {
    method: 'POST',
    headers: { 'Content-Type': 'application/json' },
    body: JSON.stringify(payload),
  });

  if (!res.ok) {
    const text = await res.text();
    throw new Error(`PayPal intent failed: ${text}`);
  }

  const data = await res.json();
  if (!data?.orderToken || !data?.paypalOrderId) {
    throw new Error('Invalid intent response');
  }

  setIntent({ orderToken: data.orderToken });
  return data.paypalOrderId as string;
}, [
  cart,
  first_name,
```

```
last_name,  
email,  
country_iso2,  
country_iso3,  
currency,  
delivery_address,  
djangoOrderIntentUuid,  
djangoOrderIntentOrderId,  
djangoOrderIntentPayload,  
djangoOrderIntentVerifyPayload,  
setIntent,  
});
```

Why this replaces `createOrderAction` in PayPal button path:

- Prevents duplicate order creation paths.
- Ensures ledger row exists before PayPal approve/capture.

15) Authorize Route Update (ledger write)

Current file:

- `src/app/api/paypal/orders/authorize/route.ts`

Add `orderToken` in request body and persist:

- `authorizePayload`
- `paypalAuthorizationId`
- `status`


```

import { NextResponse } from 'next/server';
import { OrdersController } from '@paypal/paypal-server-sdk';
import { paypalClient } from '@lib/paymentClients/paypalClient';
import { paypalTxLedger } from '@lib/prisma/shop/paypal/paypalTxLedger';
import { PAYPAL_LEDGER_STATUS } from '@lib/paypal/txLedger/status';

const orders = new OrdersController(paypalClient);

export async function POST(req: Request) {
  const { orderId, orderToken } = await req.json();
  if (!orderId || !orderToken) {
    return NextResponse.json({ error: 'orderId and orderToken required' }, { status: 400 });
  }

  try {
    const { body: resBody } = await orders.authorizeOrder({
      id: orderId,
      prefer: 'return=representation',
    });

    const authorizationId =
      resBody?.purchaseUnits?.[0]?.payments?.authorizations?.[0]?.id ??
      resBody?.purchase_units?.[0]?.payments?.authorizations?.[0]?.id;

    await paypalTxLedger.paypalIntent.update({
      where: { orderToken },
      data: {
        paypalAuthorizationId: authorizationId ?? null,
        authorizePayload: resBody as unknown as object,
        status: PAYPAL_LEDGER_STATUS.AUTHORIZED,
      },
    });

    return NextResponse.json(resBody);
  } catch (err: unknown) {
    await paypalTxLedger.paypalIntent.update({
      where: { orderToken },
      data: {
        status: PAYPAL_LEDGER_STATUS.ERROR,
        lastErrorCode: 'AUTHORIZE_FAILED',
        lastErrorMessage: err instanceof Error ? err.message : String(err),
      },
    }).catch(() => undefined);

    return NextResponse.json({ error: 'Failed to authorize' }, { status: 500 });
  }
}

```

16) Capture Route Update (ledger write)

Current file:

- `src/app/api/paypal/orders/capture/route.ts`

This section needs the same hardening mindset as Section 15. The minimal "call PayPal, then write the DB row" shape is not enough on its own, because PayPal capture can succeed while ledger persistence fails afterward.

Required outcomes:

- accept `orderToken` plus `authorizationId`
- persist `capturePayload`
- persist `status`
- reject invalid state transitions
- avoid double-capturing on retries
- return structured client-safe errors

Recommended defaults:

- use `PayPal-Request-Id` so repeated capture attempts are idempotent at PayPal's side
- if the row is already `captured` and `capturePayload` exists, return the stored payload
- if the row is `error`, only retry with the same deterministic `PayPal-Request-Id`
- if capture succeeds but ledger persistence fails, return a specific persistence error instead of pretending the whole capture failed normally

```

import { randomUUID } from 'crypto';
import { NextResponse } from 'next/server';
import { PaymentsController, type CapturedPayment } from '@paypal/paypal-server-sdk';
import { paypalClient } from '@lib/paymentClients/paypalClient';
import { paypalTxLedger } from '@lib/prisma/shop/paypal/paypalTxLedger';
import { PAYPAL_LEDGER_STATUS } from '@lib/paypal/txLedger/status';
import { createPayPalRouteResponders } from '@lib/paypal/txLedger/routeResponses';

const payments = new PaymentsController(paypalClient);

export async function POST(req: Request) {
  const requestId = randomUUID();
  let orderToken: string | undefined;

  const { error: routeError } = createPayPalRouteResponders({
    requestId,
    getOrderToken: () => orderToken,
  });

  const validationError = (code: string, message: string, status = 400) =>
    routeError({
      status,
      code,
      stage: 'validate_request',
      message,
    });

  const fail = async ({
    code,
    stage,
    message,
    err,
    status = 500,
    persistToLedger = true,
  }): {
    code: string;
    stage: string;
    message: string;
    err: unknown;
    status?: number;
    persistToLedger?: boolean;
  } => {
    console.error(`[paypal.capture.${stage}]`, {
      requestId,
      orderToken,
      code,
      error: err instanceof Error ? err.message : String(err),
      stack: err instanceof Error ? err.stack : undefined,
    });

    if (persistToLedger && orderToken) {
      await paypalTxLedger.paypalIntent.update({

```

```

        where: { orderToken },
        data: {
            status: PAYPAL_LEDGER_STATUS.ERROR,
            lastErrorCode: code,
            lastErrorMessage: err instanceof Error ? err.message : String(err),
        },
    }).catch(() => undefined);
}

return routeError({
    status,
    code,
    stage,
    message,
});
};

const persistCapturedResult = async (payload: CapturedPayment) => {
    await paypalTxLedger.paypalIntent.update({
        where: { orderToken },
        data: {
            status: PAYPAL_LEDGER_STATUS.CAPTURED,
            capturePayload: JSON.parse(JSON.stringify(payload)),
            lastErrorCode: null,
            lastErrorMessage: null,
        },
    });
};

try {
    const { authorizationId, orderToken: requestOrderToken } = await req.json();
    orderToken = requestOrderToken;

    if (!authorizationId || !orderToken) {
        return validationError(
            'INVALID_CAPTURE_REQUEST',
            'authorizationId and orderToken are required',
        );
    }

    const intent = await paypalTxLedger.paypalIntent.findUnique({
        where: { orderToken },
    });

    if (!intent) {
        return routeError({
            status: 404,
            code: 'INTENT_NOT_FOUND',
            stage: 'load_intent',
            message: 'Intent not found',
        });
    }

    if (intent.paypalAuthorizationId && intent.paypalAuthorizationId !== authorizationId) {

```

```

    return routeError({
      status: 409,
      code: 'AUTHORIZATION_MISMATCH',
      stage: 'validate_intent',
      message: 'Authorization mismatch',
    });
  }

  if (intent.status === PAYPAL_LEDGER_STATUS.CAPTURED && intent.capturePayload) {
    return Response.json(intent.capturePayload);
  }

  if (
    intent.status === PAYPAL_LEDGER_STATUS.RECEIPT_UPLOADED ||
    intent.status === PAYPAL_LEDGER_STATUS.PAYMENT_SAVED ||
    intent.status === PAYPAL_LEDGER_STATUS.COMPLETED ||
    intent.status === PAYPAL_LEDGER_STATUS.REFUNDED
  ) {
    return routeError({
      status: 409,
      code: 'CAPTURE_STATE_CONFLICT',
      stage: 'validate_intent',
      message: `Cannot capture intent from status "${intent.status}"`,
    });
  }

  if (
    intent.status !== PAYPAL_LEDGER_STATUS.AUTHORIZED &&
    intent.status !== PAYPAL_LEDGER_STATUS.ERROR
  ) {
    return routeError({
      status: 409,
      code: 'CAPTURE_STATE_INVALID',
      stage: 'validate_intent',
      message: `Unexpected intent status "${intent.status}" before capture`,
    });
  }

  const { result } = await payments.captureAuthorizedPayment({
    authorizationId,
    paypalRequestId: `capture:${orderToken}`,
    prefer: 'return=representation',
    body: { finalCapture: true },
  });

  try {
    await persistCapturedResult(result);
  } catch (persistErr) {
    return fail({
      code: 'CAPTURE_PERSIST_FAILED',
      stage: 'persist_capture_payload',
      message: 'PayPal capture succeeded but ledger persistence failed',
      err: persistErr,
      persistToLedger: false,
    });
  }
}

```

```

    });
  }

  return Response.json(result);
} catch (err: unknown) {
  return fail({
    code: 'CAPTURE_FAILED',
    stage: 'capture_payment',
    message: 'Failed to capture PayPal authorization',
    err,
  });
}
}

```

Notes:

- Capture recovery differs slightly from authorize: the safe default here is provider-side idempotency via a stable `paypalRequestId`, plus returning cached `capturePayload` when the ledger already has it.
- If you retry capture with a new request id after an ambiguous failure, you are accepting the risk of a second provider-side capture attempt.

17) Approve Hook Update (`usePayPalTXApproveCallback`)

Current file:

- `src/lib/hooks/shopHooks/checkout/usePayPalTXApproveCallback.ts`

Required changes

1. Fix parsing bug:

- remove `JSON.parse(await res.json())`

2. Pass `orderToken` from `paypalIntentStore` to authorize/capture routes.

3. Do not call client post-processing functions in this flow.

4. Redirect to the confirmation route, where polling becomes the read path.

5. Treat the client as a status viewer after capture.

- Capture still happens through your API route.
- Durable completion happens later through webhook + server runner.
- Local client state is convenience only, never the source of truth for completion.
- The canonical recovery handle is `orderToken` in the URL, not long-lived client store state.

Template fragment (core changes)

```
TS

import { usePayPalIntentStore } from '@stores/shop_stores/checkoutStore/paypalIntentStore';

const orderToken = usePayPalIntentStore((s) => s.orderToken);

// authorize
const authRes = await fetch('/next-api/paypal/orders/authorize', {
  method: 'POST',
  headers: { 'Content-Type': 'application/json' },
  body: JSON.stringify({ orderID: data.orderID, orderToken }),
});
if (!authRes.ok) {
  throw new Error(await readRouteErrorMessage(authRes, 'Authorization failed'));
}
const authData = (await authRes.json()) as OrderResponseBody;

// capture
const capRes = await fetch('/next-api/paypal/orders/capture', {
  method: 'POST',
  headers: { 'Content-Type': 'application/json' },
  body: JSON.stringify({ authorizationId, orderToken }),
});
if (!capRes.ok) {
  throw new Error(await readRouteErrorMessage(capRes, 'Capture failed'));
}
const capturedOrder = (await capRes.json()) as OrdersCapture;

// start UI, then move the browser into confirmation/status-view mode
startProcessingUI(capturedOrder.id);
router.push(`/shop/checkout/confirmation/${encodeURIComponent(orderToken)}`);
```

Minimal shared error reader:

```
TS

async function readRouteErrorMessage(res: Response, fallback: string) {
  try {
    const payload = (await res.json()) as {
      error?: { message?: string; code?: string; stage?: string; requestId?: string };
    };

    if (!payload?.error) return fallback;

    const { code, stage, requestId, message } = payload.error;
    return `${stage ?? 'unknown_stage'} ${code ?? 'UNKNOWN_ERROR'} (${requestId ?? 'no_request_id'}): ${message ?? fallback}`;
  } catch {
    return fallback;
  }
}
```

Important cleanup in this hook

When ledger flow is enabled:

- remove runtime usage of:
 - `usePost_PaymentProcessors`
 - `useServerActionWithState(...)` wrappers for receipt/payment/merchize
 - `buildServerPostProcessingFunc(...)`

Keep:

- processing UI state setters
- error toasts
- authorize/capture orchestration

Keep the hook thin:

- read the structured route error
- initialize the processing UI
- push the user to the confirmation route carrying `orderToken`
- let the confirmation page poll ledger truth

Do not make this hook responsible for receipt upload, backend save, or fulfillment retries.

If your shop runs on a dedicated domain with path normalization, use your shop-specific router abstraction here instead of raw `next/navigation` routing. The redirect target stays `/shop/checkout/confirmation/[orderToken]`, but the push helper should normalize it correctly for both the parent domain and the dedicated shop domain.

18) PayPal TX Payment Status Endpoint

New file:

- `src/app/api/paypal/tx-ledger/payments/[orderToken]/status/route.ts`

Returns ledger truth for polling.

This endpoint should be intentionally small and browser-safe:

- use `orderToken` as the lookup key
- do not treat client state as authoritative
- do not expose `paypalOrderId` unless you have a concrete browser-side need for it
- return only the fields the confirmation UI actually needs

TS

```
import { NextResponse } from 'next/server';
import { paypalTxLedger } from '@/lib/prisma/shop/paypal/paypalTxLedger';

type PageProps = {
  params: Promise<{ orderToken: string }>;
};

export async function GET(_req: Request, { params }: PageProps) {
  const { orderToken } = await params;

  const row = await paypalTxLedger.paypalIntent.findUnique({ where: { orderToken } });
  if (!row) {
    return NextResponse.json({ error: 'Not found' }, { status: 404 });
  }

  return NextResponse.json({
    orderToken: row.orderToken,
    status: row.status,
    lastEventType: row.lastEventType,
    receiptLink: row.receiptLink,
    receiptFile: row.receiptFile,
    djangoOrderIntentUuid: row.djangoOrderIntentUuid,
    djangoOrderIntentOrderId: row.djangoOrderIntentOrderId,
    djangoPaymentSaveCustomId: row.djangoPaymentSaveCustomId,
    processingCompletedAt: row.processingCompletedAt,
    error: row.lastErrorMessage
    ? { code: row.lastErrorCode, message: row.lastErrorMessage }
    : null,
  });
}
```

Optional hardening:

- If you later want to keep support correlation available, add a separate admin endpoint that can expose `paypalOrderId` server-side instead of returning it to the browser here.

19) Polling + UI Mapping (Processing Modal / Confirmation Page)

This section fills a missing piece from earlier versions: concrete polling behavior.

Polling rules

- Start polling on the confirmation page after redirect from `usePayPalTXApproveCallback`.
- Poll every 2-3 seconds.
- Stop polling when status is one of:
 - `completed`

- `error`
- `refunded`
- Also stop on unmount.
- Prefer `orderToken` from the URL if available. Treat the store as a short-lived bridge, not the recovery source.
- If the UI thinks it is "processing" but there is no `orderToken`, clear the stale local processing state instead of hanging indefinitely.

Why polling is still the default here:

- This guide assumes polling first because it is the simplest correct rollout for your current architecture.
- The confirmation client only needs occasional status changes, not a high-frequency stream.
- A 2-3 second interval is operationally cheap at low to moderate concurrency and keeps failure recovery simple.
- If you later outgrow polling, upgrade to SSE while keeping the payment-status endpoint as fallback.
- Do not choose gRPC for browser status updates in this flow. Browser support is not a good fit for a simple one-way status channel, and SSE is operationally simpler for confirmation-page style updates.

Example polling helper (confirmation-page local)

TS

```
const startLedgerPolling = (orderToken?: string) => {
  if (!orderToken) return;

  let cancelled = false;
  const interval = 2500;

  const tick = async () => {
    if (cancelled) return;
    try {
      const res = await fetch(
        `/next-api/paypal/tx-ledger/payments/${encodeURIComponent(orderToken)}/status`,
        { cache: 'no-store' },
      );
      if (!res.ok) return;
      const data = await res.json();
      applyLedgerStatusToUI(data);

      if (['completed', 'error', 'refunded'].includes(data.status)) {
        cancelled = true;
        return;
      }
    } catch {
      // swallow + retry next tick
    }
    if (!cancelled) setTimeout(tick, interval);
  };

  void tick();

  return () => {
    cancelled = true;
  };
};
```

Practical rule:

- the modal can use persisted UI state for continuity
- the confirmation page must reconcile that state against the ledger on mount
- ledger truth wins after refresh, reconnect, or browser restart

UI mapping adapter template

File:

- `src/lib/paypal/txLedger/mapLedgerToProcessingState.ts`

```
type LedgerStatusResponse = {
  status: string;
  receiptLink?: string | null;
  receiptFile?: string | null;
  error?: { code?: string | null; message?: string | null } | null;
};

export function mapLedgerToProcessingState(data: LedgerStatusResponse) {
  const status = data.status;

  if (status === 'completed') {
    return {
      flowStatus: 'completed' as const,
      steps: {
        receiptUpload: 'success',
        paymentSave: 'success',
        orderPush: 'success',
      },
    };
  }

  if (status === 'payment_saved') {
    return {
      flowStatus: 'processing' as const,
      steps: {
        receiptUpload: 'success',
        paymentSave: 'success',
        orderPush: 'pending',
      },
    };
  }

  if (status === 'receipt_uploaded') {
    return {
      flowStatus: 'processing' as const,
      steps: {
        receiptUpload: 'success',
        paymentSave: 'pending',
        orderPush: 'idle',
      },
    };
  }

  if (status === 'error') {
    return {
      flowStatus: 'error' as const,
      steps: null, // derive best-effort from last known status later if needed
    };
  }

  return {
    flowStatus: 'processing' as const,
```

```
steps: {
  receiptUpload: 'pending',
  paymentSave: 'idle',
  orderPush: 'idle',
},
};
}
```

Confirmation page integration

Current file:

- `src/app/shop/checkout/confirmation/[orderToken]/page.tsx`

The confirmation page should use `orderToken` as the route param. This keeps the browser-facing identifier aligned with the ledger key and avoids exposing `paypalOrderId` in the URL.

This is the preferred recovery model:

- the path segment is the ledger lookup key
- if the user refreshes, the page can resume from ledger truth without depending on in-memory client state

How `orderToken` reaches the confirmation page

In the approve hook (`usePayPalTXApproveCallback`), after capture succeeds, redirect with `orderToken` :

TS

```
// after successful capture in the approve hook
push(`/shop/checkout/confirmation/${encodeURIComponent(orderToken)}`);
```

Confirmation page polling template (add to existing page)

TSX

```
'use client';

import { use } from 'react';
import { useEffect, useState } from 'react';
import { mapLedgerToProcessingState } from '@lib/paypal/txLedger/mapLedgerToProcessingState';

export function useLedgerPolling({
  params,
}: {
  params: Promise<{ orderToken: string }>;
}) {
  const orderToken = use(params).orderToken;
  const [processingState, setProcessingState] = useState<ReturnType<typeof mapLedgerToProcessingState> | null>(null);

  useEffect(() => {
    if (!orderToken) return;

    let cancelled = false;
    const interval = 2500;

    const tick = async () => {
      if (cancelled) return;
      try {
        const res = await fetch(
          `/next-api/paypal/tx-ledger/payments/${encodeURIComponent(orderToken)}/status`,
          { cache: 'no-store' },
        );
        if (!res.ok) return;
        const data = await res.json();
        setProcessingState(mapLedgerToProcessingState(data));

        if (['completed', 'error', 'refunded'].includes(data.status)) {
          cancelled = true;
          return;
        }
      } catch {
        // swallow, retry next tick
      }
      if (!cancelled) setTimeout(tick, interval);
    };

    void tick();
    return () => { cancelled = true; };
  }, [orderToken]);

  return { orderToken, processingState };
}
```

Important: Do not clear checkout state immediately on mount

The confirmation page should not clear checkout/order state on first render. Clear browser stores only after a valid ledger row has been fetched, because by then the durable recovery inputs already live in Prisma (`cartSnapshot` , `shippingSnapshot` , Django intent identifiers, and PayPal payloads). The browser store is no longer the recovery source of truth after that point.

Browser-close / returning user recovery

If the user's browser closes after capture but before `completed` :

1. The ledger row already exists with status `captured` (or later).
2. The webhook will still fire and trigger `runPostProcessing` .
3. If the user returns, the confirmation page URL (bookmarked or in history) still has `orderToken` in the path — polling resumes automatically.
4. If the user lost the URL entirely, admin can look up by `customerEmail` and share status or receipt link.

No separate "check my order" page is required. The webhook guarantees completion regardless of browser state.

20) Webhook Route Upgrade (idempotent + post-processing trigger)

Current file:

- `src/app/api/paypal/webhooks/ledger-transaction-events/route.ts`

Keep your existing signature verification and raw-body handling.

Additions required

1. Delivery tracking by `PaypalWebhookEvent.eventId` plus `processedAt` / `attemptCount`
2. Robust PayPal order id extraction
3. Retry when the ledger row is not yet correlated
4. `runPostProcessing(orderToken)` trigger on `PAYMENT.CAPTURE.COMPLETED`
5. ACK fast; do not let the webhook route become the place where long receipt/backend/fulfillment work blocks the provider response

Why `after()` for post-processing (Next.js 15+)

PayPal expects a response within ~10 seconds. If `runPostProcessing` takes longer (receipt upload, Merchize API, payment save), the webhook times out and PayPal marks the delivery as failed — triggering retries that overlap with your still-running work.

`after()` from `next/server` solves this: the webhook handler responds 200 immediately, and the post-processing callback runs in the same Node.js process after the response is sent. Your VPS runs Node.js directly (not edge/serverless), so `after()` is fully supported.

Tradeoff: since we respond 200 before post-processing finishes, PayPal considers the delivery successful. If the Node process crashes mid-`after()`, PayPal will not retry. The webhook event stays in `received` status — recover via admin retry (Section 24) or a periodic recovery query:

TS

```
// Find webhook events stuck in 'received' for more than 10 minutes
const stale = await paypalTxLedger.paypalWebhookEvent.findMany({
  where: {
    processingStatus: 'received',
    createdAt: { lt: new Date(Date.now() - 10 * 60 * 1000) },
  },
});
```

This tradeoff is standard in production webhook systems — ACK fast, process async, recover from your own state.

Robust PayPal webhook correlation helper

PayPal event shapes vary. Prefer the real PayPal order ID when it is present. If PayPal only returns `custom_id`, treat that value as the local `orderToken` because the create-order payload sets `customId: orderToken`.

TS

```
type WebhookCorrelation =
  | { source: 'paypal_order_id'; paypalOrderId: string }
  | { source: 'order_token'; orderToken: string };

function getWebhookLedgerCorrelation(event: any): WebhookCorrelation | undefined {
  const paypalOrderId = event?.resource?.supplementary_data?.related_ids?.order_id;
  if (paypalOrderId) return { source: 'paypal_order_id', paypalOrderId };

  const orderToken = event?.resource?.custom_id || event?.resource?.invoice_id;
  if (orderToken) return { source: 'order_token', orderToken };

  return undefined;
}
```

Note:

- For your flow, primary path is still `supplementary_data.related_ids.order_id`.
- `custom_id` is now expected to be the local `orderToken`, so lookup by `orderToken` when that fallback is used.
- Keep `paypalOrderId`, `orderToken`, and Django `custom_id` separate in naming and UI labels.
- Do not use unrelated fields as fake fallbacks just to avoid `undefined`. If correlation fails, record that explicitly and let the webhook retry or recovery path handle it.

Default idempotent webhook core template (merge into your route)

TS

```
import { after } from 'next/server';
import { paypalTxLedger } from '@lib/prisma/shop/paypal/paypalTxLedger';
import { PAYPAL_LEDGER_STATUS } from '@lib/paypal/txLedger/status';
import { runPostProcessing } from '@lib/paypal/txLedger/runPostProcessing';

// after verification + parsed `event`
const eventId = event.id;
const eventType = event.event_type;
const correlation = getWebhookLedgerCorrelation(event);

if (!eventId) return new Response('Missing event id', { status: 400 });

let webhookEvent = await paypalTxLedger.paypalWebhookEvent.findUnique({
  where: { eventId },
});

if (!webhookEvent) {
  webhookEvent = await paypalTxLedger.paypalWebhookEvent.create({
    data: {
      eventId,
      eventType,
      paypalOrderId: correlation?.source === 'paypal_order_id' ? correlation.paypalOrderId : null,
      payload: event,
      processingStatus: 'received',
    },
  });
}

if (webhookEvent.processedAt) {
  return new Response('OK', { status: 200 });
}

if (!correlation) {
  await paypalTxLedger.paypalWebhookEvent.update({
    where: { eventId },
    data: {
      processingStatus: 'ignored',
      processedAt: new Date(),
      lastAttemptAt: new Date(),
      attemptCount: { increment: 1 },
      lastErrorMessage: null,
    },
  });

  return new Response('OK', { status: 200 });
}

const row = await paypalTxLedger.paypalIntent.findUnique({
  where:
    correlation.source === 'paypal_order_id'
```

```

        ? { paypalOrderId: correlation.paypalOrderId }
        : { orderToken: correlation.orderToken },
    });
    if (!row) {
        await paypalTxLedger.paypalWebhookEvent.update({
            where: { eventId },
            data: {
                processingStatus: 'awaiting_ledger_row',
                lastAttemptAt: new Date(),
                attemptCount: { increment: 1 },
                lastErrorMessage: 'Ledger row not correlated yet',
            },
        });

        return new Response('Retry later', { status: 500 });
    }

    if (row.status === PAYPAL_LEDGER_STATUS.COMPLETED) {
        await paypalTxLedger.paypalWebhookEvent.update({
            where: { eventId },
            data: {
                processingStatus: 'processed',
                processedAt: new Date(),
                lastAttemptAt: new Date(),
                attemptCount: { increment: 1 },
                lastErrorMessage: null,
            },
        });

        return new Response('OK', { status: 200 }); // never rerun completion
    }

    try {
        switch (eventType) {
            case 'PAYMENT.AUTHORIZATION.CREATED':
                await paypalTxLedger.paypalIntent.update({
                    where: { paypalOrderId },
                    data: { status: PAYPAL_LEDGER_STATUS.AUTHORIZED, lastEventType: eventType },
                });
                break;

            case 'PAYMENT.CAPTURE.PENDING':
                await paypalTxLedger.paypalIntent.update({
                    where: { paypalOrderId },
                    data: { status: PAYPAL_LEDGER_STATUS.PENDING, lastEventType: eventType },
                });
                break;

            case 'PAYMENT.CAPTURE.DENIED':
                await paypalTxLedger.paypalIntent.update({
                    where: { paypalOrderId },
                    data: {
                        status: PAYPAL_LEDGER_STATUS.ERROR,
                        lastEventType: eventType,

```

```

        lastErrorCode: 'CAPTURE_DENIED',
        lastErrorMessage: 'PayPal capture denied',
    },
});
break;

case 'PAYMENT.CAPTURE.REFUNDED':
    await paypalTxLedger.paypalIntent.update({
        where: { paypalOrderId },
        data: { status: PAYPAL_LEDGER_STATUS.REFUNDED, lastEventType: eventType },
    });
    break;

case 'PAYMENT.CAPTURE.COMPLETED':
    if (
        ![
            PAYPAL_LEDGER_STATUS.RECEIPT_UPLOADED,
            PAYPAL_LEDGER_STATUS.PAYMENT_SAVED,
            PAYPAL_LEDGER_STATUS.COMPLETED,
        ].includes(row.status)
    ) {
        await paypalTxLedger.paypalIntent.update({
            where: { paypalOrderId },
            data: { status: PAYPAL_LEDGER_STATUS.CAPTURED, lastEventType: eventType },
        });
    }

    // Schedule post-processing to run AFTER the response is sent to PayPal.
    // This prevents webhook timeouts under load. The DB-backed lease in
    // runPostProcessing prevents duplicate work if a retry overlaps.
    after(async () => {
        try {
            await runPostProcessing(row.orderToken);

            await paypalTxLedger.paypalWebhookEvent.update({
                where: { eventId },
                data: {
                    processingStatus: 'processed',
                    processedAt: new Date(),
                    lastAttemptAt: new Date(),
                    attemptCount: { increment: 1 },
                    lastErrorMessage: null,
                },
            });
        } catch (err) {
            const message = err instanceof Error ? err.message : String(err);

            await paypalTxLedger.paypalWebhookEvent.update({
                where: { eventId },
                data: {
                    processingStatus: 'failed',
                    lastAttemptAt: new Date(),
                    attemptCount: { increment: 1 },
                    lastErrorMessage: message,
                },
            });
        }
    });
}

```

```

    },
    });

    await paypalTxLedger.paypalIntent.update({
      where: { paypalOrderId },
      data: {
        lastErrorCode: 'POST_PROCESSING_FAILED',
        lastErrorMessage: message,
      },
    });
  }
});

// Respond 200 immediately. Post-processing continues via after().
// If the process crashes before after() completes, the webhook event
// stays in 'received' status – recover via admin retry (Section 24).
return new Response('OK', { status: 200 });
}

await paypalTxLedger.paypalWebhookEvent.update({
  where: { eventId },
  data: {
    processingStatus: 'processed',
    processedAt: new Date(),
    lastAttemptAt: new Date(),
    attemptCount: { increment: 1 },
    lastErrorMessage: null,
  },
});
} catch (err) {
  const message = err instanceof Error ? err.message : String(err);

  await paypalTxLedger.paypalWebhookEvent.update({
    where: { eventId },
    data: {
      processingStatus: 'failed',
      lastAttemptAt: new Date(),
      attemptCount: { increment: 1 },
      lastErrorMessage: message,
    },
  });

  await paypalTxLedger.paypalIntent.update({
    where: { paypalOrderId },
    data: {
      lastErrorCode: 'POST_PROCESSING_FAILED',
      lastErrorMessage: message,
    },
  });

  return new Response('Webhook processing failed', { status: 500 });
}

return new Response('OK', { status: 200 });

```

Operational rule:

- the webhook is the durable completion trigger
- the browser is not part of that trust boundary
- if the client disappears entirely after capture, the webhook path must still be sufficient to complete the order

21) Server Post-Processing Runner (ledger-driven)

This is the core server-only replacement for client post-processing.

New file:

- `src/lib/paypal/txLedger/runPostProcessing.ts`

Important:

- This section is ledger-focused.
- Provider-specific fulfillment logic is delegated (see Section 22).
- The runner must be restart-safe and retry-safe.
- The runner must be able to finish the order from ledger state alone, without any browser-only context.

Important: server-only crypto

The existing `encrypt` helper currently lives in `@/stores/shop_stores/cartStore`, which is a `'use client'` file and depends on a public env var. Do **not** reuse that helper from webhook/server code.

Keep two helpers:

- client-side checkout encryption can stay on its current path until you refactor the OTP flow
- webhook/post-processing must use a server-only helper backed by `SHOP_SERVER_ACTIONS_CRYPT0_SECRET`

TS

```
// src/lib/utils/shop/checkout/serverPostProcessingCrypto.ts
import 'server-only';
import CryptoJS from 'crypto-js';

function getServerActionSecret() {
  const secret = process.env.SHOP_SERVER_ACTIONS_CRYPT0_SECRET;
  if (!secret) {
    throw new Error('SHOP_SERVER_ACTIONS_CRYPT0_SECRET is not configured');
  }
  return secret;
}

export function encryptForPostProcessingServerAction(text: string): string {
  return CryptoJS.AES.encrypt(text, getServerActionSecret()).toString();
}
```

Update the decrypt side of `savePaymentReceiptToCloud`, `savePaymentDataToBackend`, and fulfillment server actions to use the same server-only secret. Do not import this module into client code.

```

import { randomUUID } from 'crypto';
import { encryptForPostProcessingServerAction } from
'@/lib/utils/shop/checkout/serverPostProcessingCrypto';
import { paypalTxLedger } from '@/lib/prisma/shop/paypal/paypalTxLedger';
import { PAYPAL_LEDGER_STATUS } from '@/lib/paypal/txLedger/status';
import { savePaymentReceiptToCloud } from
'@/actions/shop/paypal/processAndUploadCompletedTx/savePaymentReceiptToCloud';
import { savePaymentDataToBackend } from
'@/actions/shop/paypal/processAndUploadCompletedTx/savePaymentDataToBackend';
import { sendMerchizeFulfillmentOrder } from '@/lib/paypal/txLedger/sendMerchizeFulfillmentOrder';

const POST_PROCESSING_LEASE_MS = 5 * 60_000;

export async function runPostProcessing(orderToken: string) {
  const now = new Date();
  const leaseExpiresAt = new Date(now.getTime() + POST_PROCESSING_LEASE_MS);
  const lockId = randomUUID();

  const lease = await paypalTxLedger.paypalIntent.updateMany({
    where: {
      orderToken,
      processingCompletedAt: null,
      OR: [
        { postProcessingLockExpiresAt: null },
        { postProcessingLockExpiresAt: { lt: now } },
      ],
    },
    data: {
      postProcessingLockId: lockId,
      postProcessingLockedAt: now,
      postProcessingLockExpiresAt: leaseExpiresAt,
    },
  });

  if (lease.count === 0) return;

  const row = await paypalTxLedger.paypalIntent.findUnique({ where: { orderToken } });
  if (!row) throw new Error('Ledger row not found');

  if (row.postProcessingLockId !== lockId) return;

  if (row.processingCompletedAt || row.status === PAYPAL_LEDGER_STATUS.COMPLETED) {
    await paypalTxLedger.paypalIntent.updateMany({
      where: { orderToken, postProcessingLockId: lockId },
      data: {
        postProcessingLockId: null,
        postProcessingLockedAt: null,
        postProcessingLockExpiresAt: null,
      },
    });
    return;
  }
}

```

```
try {
  const authData = row.authorizePayload as any;
  const finalCapturedOrder = row.capturePayload as any;
  if (!authData || !finalCapturedOrder) {
    throw new Error('Missing authorize/capture payload in ledger');
  }

  const customer = { name: row.customerName, email: row.customerEmail };
  const ORD_string = row.djangoOrderIntentOrderId ?? row.orderToken;

  // Step 1: Receipt
  if (!row.receiptLink || !row.receiptFile) {
    const receiptPayload = encryptForPostProcessingServerAction(JSON.stringify({ authData, customer,
ORD_string }));
    const receiptRes = await savePaymentReceiptToCloud(receiptPayload);
    if (!receiptRes.success) {
      throw new Error(receiptRes.message ?? 'Receipt upload failed');
    }

    await paypalTxLedger.paypalIntent.updateMany({
      where: { orderToken, postProcessingLockId: lockId },
      data: {
        status: PAYPAL_LEDGER_STATUS.RECEIPT_UPLOADED,
        receiptLink: receiptRes.pdfReceiptLink,
        receiptFile: receiptRes.receiptFileName,
        lastErrorCode: null,
        lastErrorMessage: null,
      },
    });

    row.receiptLink = receiptRes.pdfReceiptLink;
    row.receiptFile = receiptRes.receiptFileName;
  }

  // Step 2: Payment save
  if (!row.djangoPaymentSaveCustomId) {
    const savePayload = encryptForPostProcessingServerAction(
      JSON.stringify({
        authData,
        finalCapturedOrder,
        capturedOrderPaypalID: row.paypalOrderId,
        customer,
        delivery_address: row.shippingSnapshot,
        userId: row.userId,
        ORD_string,
        pdfReceiptLink: row.receiptLink,
        receiptFileName: row.receiptFile,
      })),
    );

    const paymentSave = await savePaymentDataToBackend(savePayload);
    if (!paymentSave.ok) {
      throw new Error(paymentSave.error?.message ?? 'Payment save failed');
```

```

    }

    await paypalTxLedger.paypalIntent.updateMany({
      where: { orderToken, postProcessingLockId: lockId },
      data: {
        status: PAYPAL_LEDGER_STATUS.PAYMENT_SAVED,
        djangoPaymentSaveCustomId: paymentSave.data?.custom_id ?? null,
        lastErrorCode: null,
        lastErrorMessage: null,
      },
    });

    row.djangoPaymentSaveCustomId = paymentSave.data?.custom_id ?? null;
  }

  if (!row.djangoPaymentSaveCustomId) {
    throw new Error('Missing Django payment save custom_id before fulfillment push');
  }

  // Step 3: Merchize fulfillment
  await sendMerchizeFulfillmentOrder({
    cartSnapshot: row.cartSnapshot,
    djangoPaymentSaveCustomId: row.djangoPaymentSaveCustomId,
    identifier: 'codexchristi-shop',
    currency: row.initialCurrency ?? 'USD',
    customerName: row.customerName,
    fulfillmentAddress: row.shippingSnapshot,
  });

  await paypalTxLedger.paypalIntent.updateMany({
    where: { orderToken, postProcessingLockId: lockId },
    data: {
      status: PAYPAL_LEDGER_STATUS.COMPLETED,
      processingCompletedAt: new Date(),
      postProcessingLockId: null,
      postProcessingLockedAt: null,
      postProcessingLockExpiresAt: null,
      lastErrorCode: null,
      lastErrorMessage: null,
    },
  });
} catch (err) {
  const message = err instanceof Error ? err.message : String(err);

  await paypalTxLedger.paypalIntent.updateMany({
    where: { orderToken, postProcessingLockId: lockId },
    data: {
      lastErrorCode: 'POST_PROCESSING_FAILED',
      lastErrorMessage: message,
      postProcessingLockId: null,
      postProcessingLockedAt: null,
      postProcessingLockExpiresAt: null,
    },
  });
}

```



```
    throw err;
  }
}
```

Operational note:

- If receipt upload, payment save, or fulfillment can exceed the lease duration, renew `postProcessingLockExpiresAt` between steps.
- Pass `orderToken` as the downstream idempotency key wherever the target API supports it.
- Treat the ledger row as the single source of truth for what still needs to run. Do not rebuild required inputs from client stores at this stage.
- Each step should be safely skippable if its success artifact already exists in the ledger row.

Fulfillment validation and provider response handling:

- Build the canonical Merchize/Django process payload before sending anything to Django.
- Normalize `country` to ISO2 at the fulfillment boundary. Checkout/pricing may store ISO3 (`USA`), but `/orders/process/{custom_id}` must receive the provider-compatible value (`US`).
- Validate required local payload fields before the provider call. Expected validation failures are not process crashes; they are paid-order recovery states.
- On local validation failure:
 - Set `status = fulfillment_blocked`.
 - Set `lastErrorCode = FULFILLMENT_PAYLOAD_INVALID`.
 - Set `lastErrorMessage` to a concise field-level summary.
 - Save `merchizeFulfillmentRequestPayload` with the canonical attempted payload.
 - Save `merchizeFulfillmentResponsePayload` as a local validation envelope, for example `{ source: "local_validation", success: false, issues: [...] }`.
 - Clear the post-processing lease.
 - Enqueue an `AdminNotificationOutbox` row with a deterministic `dedupeKey`.
- A HTTP 2xx response from Django is only transport success. It is not fulfillment success.
- On Django business failure, such as `{ success: false }` or `data.processing_status === "failed"`:
 - Set `status = fulfillment_failed`.
 - Set `lastErrorCode = FULFILLMENT_PROVIDER_REJECTED`.
 - Set `lastErrorMessage` from `data.error_message ?? message`.
 - Save both fulfillment request and response payloads.
 - Clear the post-processing lease.
 - Enqueue an admin notification outbox row.
- Only set `status = completed` and `processingCompletedAt` after receipt upload, Django payment save, and fulfillment business success have all been confirmed.

22) Merchize Fulfillment Helper

This section isolates Merchize fulfillment payload construction from the main ledger runner.

Current file:

- `src/lib/paypal/txLedger/sendMerchizeFulfillmentOrder.ts`

Ledger-facing helper shape

```
TS

// src/lib/paypal/txLedger/sendMerchizeFulfillmentOrder.ts
import { sendMerchizeOrderDetailsToBackend } from
'@/actions/shop/checkout/createMerchizeOrder/sendMerchizeOrderDetailsToBackend';
import { encryptForPostProcessingServerAction } from
'@/lib/utils/shop/checkout/serverPostProcessingCrypto';

export type MerchizeFulfillmentOrderArgs = {
  cartSnapshot: CartVariant[];
  djangoPaymentSaveCustomId: string;
  identifier: string;
  currency: string;
  customerName: string;
  fulfillmentAddress: PaymentSavingActionProps['delivery_address'];
};

export async function sendMerchizeFulfillmentOrder(args: MerchizeFulfillmentOrderArgs) {
  const requestPayload = buildFulfillmentPayload(
    args,
    mapCartToProcessingItems(args.cartSnapshot, args.currency),
  );

  const response = await sendMerchizeOrderDetailsToBackend(
    encryptForPostProcessingServerAction(
      JSON.stringify({
        djangoPaymentSaveCustomId: args.djangoPaymentSaveCustomId,
        payload: requestPayload,
      }),
    ),
  );

  if (!response.ok) throw new Error(response.error?.message ?? 'Fulfillment push failed');
  return { ...response, requestPayload };
}
```

Why this matters:

- The Django payment save custom ID is the fulfillment process path identifier.
- The fulfillment request payload is saved back to the ledger as `merchizeFulfillmentRequestPayload`.
- The fulfillment response payload is saved back to the ledger as `merchizeFulfillmentResponsePayload`.

Required `/orders/process/{custom_id}` body shape:

TS

```
type OrderProcessingRequest = {
  identifier: string;
  items: Array<{
    product_id: string;
    sku?: string;
    merchize_sku: string;
    quantity: number;
    price: number;
    currency: string;
    image: string;
  }>;
  first_name?: string;
  last_name?: string;
  address?: string;
  address_2?: string;
  city?: string;
  state?: string;
  zip_code?: string;
  country?: string; // ISO2 for the current Django/Merchize process contract
  phone?: string;
};
```

Business success predicate:

TS

```
function isFulfillmentBusinessSuccess(response: OrderProcessingAPIResponse) {
  return (
    response.success === true &&
    response.data?.processing_status !== 'failed' &&
    !response.data?.error_message
  );
}
```

Implementation rules:

- `sendMerchizeOrderDetailsToBackend(...)` must return `ok: false` when the HTTP request succeeds but Django returns `success: false`, `processing_status: "failed"`, or `error_message`.
- The helper should expose the raw Django response in the returned error object so `runPostProcessing(...)` can persist it.
- Do not throw away provider response data just because the normalized result is `ok: false`.
- Do not mark the ledger row `completed` from HTTP status alone.
- For provider-accepted but asynchronous states, decide explicitly which `processing_status` values count as accepted. Treat unknown states as `FULFILLMENT_PROVIDER_REJECTED` until the Django contract is confirmed.

23) Admin Auth (header secret)

Decision default:

- Header-only auth for now (cookie/session auth is an optional upgrade — see Section 31).
- No cookie/session logic in these templates.

New file:

- `src/lib/utils/adminAuth.ts`

TS

```
export function assertAdminSecret(req: Request) {
  const provided = req.headers.get('x-admin-secret');
  const expected = process.env.ADMIN_SECRET_KEY;

  if (!expected) {
    throw new Error('ADMIN_SECRET_KEY is not configured');
  }

  if (!provided || provided !== expected) {
    const err = new Error('Unauthorized');
    (err as Error & { status?: number }).status = 401;
    throw err;
  }
}
```

24) Admin Ledger UI + Retry Actions (Search, Filter, Retry)

New files:

- `src/app/admin/shop/order-recovery/page.tsx`
- `src/app/admin/shop/order-recovery/actions.ts`
- `src/app/admin/shop/order-recovery/AdminOrderRecoveryLedgerTable.tsx`

Goals for the admin UI

- Search by:
 - `orderToken`
 - `paypalOrderId`
 - `customerEmail`
 - `djangoOrderIntentUuid`
 - `djangoOrderIntentOrderId`
 - `djangoPaymentSaveCustomId`
- Filter by:
 - `status`
 - date window (optional simple version)
- Retry:

- full post-processing (`receipt -> payment save -> fulfillment`)
- later (optional) step-specific retries
- Inspect:
 - last error code/message
 - retry count
 - created/updated timestamps
 - fulfillment validation issues
 - provider business failure response
 - admin notification outbox status/history

Server actions template

TS

```
// src/app/admin/shop/order-recovery/actions.ts
'use server';

import { paypalTxLedger } from '@lib/prisma/shop/paypal/paypalTxLedger';
import { runPostProcessing } from '@lib/paypal/txLedger/runPostProcessing';

export async function retryPostProcessing(orderToken: string) {
  const row = await paypalTxLedger.paypalIntent.findUnique({ where: { orderToken } });
  if (!row) throw new Error('Ledger row not found');

  await paypalTxLedger.paypalIntent.update({
    where: { orderToken },
    data: { retryCount: { increment: 1 } },
  });

  await runPostProcessing(orderToken);
}
```

Notification and retry rules:

- Admin notification rows are created when a transaction reaches a valid internal-alert event.
- The admin UI should show whether an internal alert is pending, sent, failed, or suppressed.
- Retry actions should not create unlimited duplicate alerts. Use an outbox `dedupeKey` based on order token, failure type, and current failing stage.
- Fulfillment-only retry should rebuild and revalidate the fulfillment payload before sending to Django.
- If retry succeeds, leave the previous outbox row for audit and optionally create a low-severity recovery notification.

Valid internal-alert events:

- `PAYPAL_INTENT_FAILED`: PayPal order creation failed after a ledger row exists.
- `PAYPAL_AUTHORIZE_FAILED`: authorization failed or returned an unusable payload.
- `PAYPAL_CAPTURE_FAILED`: capture failed, was denied, or returned an ambiguous state needing review.
- `POST_PROCESSING_FAILED`: receipt upload, Django payment save, or another non-fulfillment post-processing step failed.
- `FULFILLMENT_PAYLOAD_INVALID`: local fulfillment payload validation blocked a paid order before the provider call.

- `FULFILLMENT_PROVIDER_REJECTED` : Django/Merchize returned transport success but business failure, including HTTP 200 with `success: false`, `processing_status: "failed"`, or `error_message`.
- `PAID_ROW_STUCK` : a captured row remains unresolved beyond the configured recovery window.
- `ADMIN_RETRY_FAILED` : an admin-triggered retry failed and still needs review.
- `ADMIN_RETRY_RECOVERED` : an admin-triggered retry completed a previously unresolved paid order; use low severity.

Email content rules:

- Include support reference, order token, PayPal order ID when available, stage, error code, concise message, current status, and admin detail link.
- Include a redacted issue summary for PII-heavy data. Do not put full raw PayPal/Django/Merchize payloads in email.
- Link to `/admin/shop/order-recovery/[orderToken]` for full payload inspection.
- Use one outbox row per recipient so delivery/retry state is recipient-specific.

Query helpers for search/filter (server-side)

This is the missing practical part: the admin page should not fetch everything unfiltered.

```
// add to src/app/admin/shop/order-recovery/actions.ts (or a local server helper file)
'use server';

import { paypalTxLedger } from '@lib/prisma/shop/paypal/paypalTxLedger';

type ListLedgerArgs = {
  q?: string;
  status?: string;
  page?: number;
  pageSize?: number;
};

export async function listLedgerRows({
  q,
  status,
  page = 1,
  pageSize = 25,
}: ListLedgerArgs) {
  const safePage = Math.max(1, page);
  const safePageSize = Math.min(100, Math.max(10, pageSize));
  const skip = (safePage - 1) * safePageSize;

  const where = {
    ...(status ? { status } : {}),
    ...(q
      ? {
          OR: [
            { orderToken: { contains: q, mode: 'insensitive' as const } },
            { paypalOrderId: { contains: q, mode: 'insensitive' as const } },
            { customerEmail: { contains: q, mode: 'insensitive' as const } },
            { djangoOrderIntentUuid: { contains: q, mode: 'insensitive' as const } },
            { djangoOrderIntentOrderId: { contains: q, mode: 'insensitive' as const } },
            { djangoPaymentSaveCustomId: { contains: q, mode: 'insensitive' as const } },
          ],
        }
      : {}),
  };

  const [rows, total] = await Promise.all([
    paypalTxLedger.paypalIntent.findMany({
      where,
      orderBy: { createdAt: 'desc' },
      skip,
      take: safePageSize,
    }),
    paypalTxLedger.paypalIntent.count({ where }),
  ]);

  return {
    rows,
    total,
    page: safePage,
  };
}
```

```
    pageSize: safePageSize,  
    totalPages: Math.max(1, Math.ceil(total / safePageSize)),  
  };  
}
```


Admin page route template (server component shell)

TSX

```
// src/app/admin/shop/order-recovery/page.tsx
import AdminLedgerTable from './AdminLedgerTable';
import { listLedgerRows } from './actions';

type PageProps = {
  searchParams?: Promise<{
    q?: string;
    status?: string;
    page?: string;
  }>;
};

export default async function PayPalLedgerAdminPage({ searchParams }: PageProps) {
  const params = (await searchParams) ?? {};
  const q = params.q?.trim() || '';
  const status = params.status?.trim() || '';
  const page = Number(params.page || '1');

  const data = await listLedgerRows({
    q: q || undefined,
    status: status || undefined,
    page: Number.isFinite(page) ? page : 1,
    pageSize: 25,
  });

  return (
    <main className='p-6 space-y-6'>
      <header className='space-y-1'>
        <h1 className='text-2xl font-semibold'>PayPal TX Ledger</h1>
        <p className='text-sm opacity-80'>
          Search, inspect, and retry incomplete or failed post-payment processing.
        </p>
      </header>

      <AdminLedgerTable
        rows={data.rows}
        total={data.total}
        page={data.page}
        totalPages={data.totalPages}
        q={q}
        status={status}
      />
    </main>
  );
}
```

Admin table UI template (search + filter + retry)

TSX

```
// src/app/admin/shop/order-recovery/AdminOrderRecoveryLedgerTable.tsx
'use client';

import { useMemo, useTransition } from 'react';
import { useRouter, useSearchParams } from 'next/navigation';
import { retryPostProcessing } from './actions';

type LedgerRow = {
  id: string;
  orderToken: string;
  paypalOrderId: string | null;
  customerEmail: string;
  djangoOrderIntentUuid: string | null;
  djangoOrderIntentOrderId: string | null;
  djangoPaymentSaveCustomId: string | null;
  status: string;
  retryCount: number;
  lastErrorCode: string | null;
  lastErrorMessage: string | null;
  createdAt: Date;
  updatedAt: Date;
};

type Props = {
  rows: LedgerRow[];
  total: number;
  page: number;
  totalPages: number;
  q: string;
  status: string;
};

const STATUS_OPTIONS = [
  '',
  'intent_creating',
  'intent_created',
  'authorized',
  'captured',
  'receipt_uploaded',
  'payment_saved',
  'pending',
  'completed',
  'refunded',
  'error',
];

export default function AdminLedgerTable({
  rows,
  total,
  page,
```

```

totalPages,
q,
status,
}): Props) {
  const router = useRouter();
  const searchParams = useSearchParams();
  const [isPending, startTransition] = useTransition();

  const setQuery = (next: { q?: string; status?: string; page?: number }) => {
    const params = new URLSearchParams(searchParams?.toString() ?? '');
    if (next.q !== undefined) {
      next.q ? params.set('q', next.q) : params.delete('q');
    }
    if (next.status !== undefined) {
      next.status ? params.set('status', next.status) : params.delete('status');
    }
    if (next.page !== undefined) {
      next.page > 1 ? params.set('page', String(next.page)) : params.delete('page');
    }
    startTransition(() => {
      router.replace(`/admin/shop/order-recovery?${params.toString()}`);
    });
  };

  const canPrev = page > 1;
  const canNext = page < totalPages;

  const summary = useMemo(
    () => `Showing ${rows.length} of ${total} total records`,
    [rows.length, total],
  );

  return (
    <section className='space-y-4'>
      <form
        className='grid gap-3 md:grid-cols-[1fr_220px_auto]'
        onSubmit={(e) => {
          e.preventDefault();
          const fd = new FormData(e.currentTarget);
          setQuery({
            q: String(fd.get('q') || ''),
            status: String(fd.get('status') || ''),
            page: 1,
          });
        }}
      >
        <input
          name='q'
          defaultValue={q}
          placeholder='Search by orderToken / PayPal ID / email / Django IDs'
          className='border rounded px-3 py-2'
        />
        <select name='status' defaultValue={status} className='border rounded px-3 py-2'>
          {STATUS_OPTIONS.map((s) => (

```

```

        <option key={s || 'all'} value={s}>
          {s || 'All statuses'}
        </option>
      )}}
    </select>
    <button type='submit' className='border rounded px-4 py-2'>
      {isPending ? 'Searching...' : 'Search'}
    </button>
  </form>

  <div className='text-sm opacity-80'>{summary}</div>

  <div className='overflow-x-auto border rounded'>
    <table className='w-full text-sm'>
      <thead className='bg-black/5'>
        <tr>
          <th className='text-left p-2'>Status</th>
          <th className='text-left p-2'>Customer</th>
          <th className='text-left p-2'>Order Token</th>
          <th className='text-left p-2'>PayPal Order</th>
          <th className='text-left p-2'>Django Intent</th>
          <th className='text-left p-2'>Django Payment Save ID</th>
          <th className='text-left p-2'>Retries</th>
          <th className='text-left p-2'>Last Error</th>
          <th className='text-left p-2'>Actions</th>
        </tr>
      </thead>
      <tbody>
        {rows.map((row) => (
          <tr key={row.id} className='border-t align-top'>
            <td className='p-2'>{row.status}</td>
            <td className='p-2'>{row.customerEmail}</td>
            <td className='p-2 break-all'>{row.orderToken}</td>
            <td className='p-2 break-all'>{row.paypalOrderId ?? '-'}</td>
            <td className='p-2 break-all'>
              <div>{row.djangoOrderIntentOrderId ?? '-'}</div>
              <div className='opacity-70'>{row.djangoOrderIntentUuid ?? '-'}</div>
            </td>
            <td className='p-2 break-all'>{row.djangoPaymentSaveCustomId ?? '-'}</td>
            <td className='p-2'>{row.retryCount}</td>
            <td className='p-2 max-w-[320px]'>
              <div className='font-medium'>{row.lastErrorCode ?? '-'}</div>
              <div className='opacity-80 break-words'>{row.lastErrorMessage ?? '-'}</div>
            </td>
            <td className='p-2'>
              <form action={retryPostProcessing.bind(null, row.orderToken)}>
                <button
                  type='submit'
                  className='border rounded px-3 py-1'
                  disabled={row.status === 'completed'}
                >
                  Retry Post-Processing
                </button>
              </form>
            </td>
          </tr>
        ))}
      </tbody>
    </table>
  </div>

```

```

        </td>
      </tr>
    )}}
    {rows.length === 0 && (
      <tr>
        <td className='p-4 text-center opacity-70' colSpan={9}>
          No ledger rows found for current filters.
        </td>
      </tr>
    )}
  </tbody>
</table>
</div>

<div className='flex items-center justify-between gap-3'>
  <button
    type='button'
    className='border rounded px-3 py-2 disabled:opacity-50'
    disabled={!canPrev || isPending}
    onClick={() => setQuery({ page: page - 1 })}
  >
    Previous
  </button>
  <div className='text-sm'>
    Page {page} of {totalPages}
  </div>
  <button
    type='button'
    className='border rounded px-3 py-2 disabled:opacity-50'
    disabled={!canNext || isPending}
    onClick={() => setQuery({ page: page + 1 })}
  >
    Next
  </button>
</div>
</section>
);
}

```

Optional improvements (safe to add later)

- Step-specific retry buttons:
 - `Retry Receipt Only`
 - `Retry Payment Save + Fulfillment`
 - `Retry Fulfillment Only`
- JSON payload inspector drawer for authorize/capture payloads
- CSV export for filtered results
- date-range filtering

Note:

- If you want route-level auth later, add a protected wrapper route or middleware.

- Can also be restricted by reverse proxy/private network.

25) Sandbox/Dev Fallback Trigger

Why:

- PayPal sandbox webhook delivery/testing can be inconsistent depending on environment and simulator behavior.
- You still need to validate the ledger post-processing path.

New file:

- `src/app/api/dev/paypal/replay-post-processing/route.ts`

TS

```
import { NextResponse } from 'next/server';
import { assertAdminSecret } from '@lib/utils/adminAuth';
import { runPostProcessing } from '@lib/paypal/txLedger/runPostProcessing';

export async function POST(req: Request) {
  if (process.env.NODE_ENV === 'production') {
    return NextResponse.json({ error: 'Not allowed in production' }, { status: 403 });
  }

  try {
    assertAdminSecret(req);
  } catch (err) {
    return NextResponse.json({ error: 'Unauthorized' }, { status: 401 });
  }

  const { orderToken } = await req.json();
  if (!orderToken) {
    return NextResponse.json({ error: 'Missing orderToken' }, { status: 400 });
  }

  await runPostProcessing(orderToken);
  return NextResponse.json({ ok: true });
}
```

This endpoint is a **dev-only tool**, not part of production payment correctness.

26) Rollout and Migration Order (safe deployment)

This is the missing operational sequence from earlier drafts.

Step 1: Database foundation

1. Add schema file.
2. Add `AdminNotificationOutbox`.
3. Run `prisma format` + `prisma validate`.
4. Run ledger migrations.
5. Generate ledger Prisma client.

Step 2: Write-only ledger instrumentation (no behavior change yet)

1. Add ledger client wrapper.
2. Add status constants/types.
3. Add intent endpoint.
4. Update `PayPalCheckoutChildren` to create intents and store `orderToken`.
5. Update authorize/capture routes to write ledger payloads and statuses.

At this point:

- Existing checkout behavior should still work.
- Authorize/capture still happen synchronously through your existing API routes.
- The client is not the durable owner of completion after capture.
- Client post-processing remains disabled as currently coded.

Step 3: Read path + UI sync

1. Add order status endpoint.
2. Add polling and UI mapping.
3. Verify processing UI updates from ledger.

Step 4: Durable completion path

1. Add webhook idempotency and status transitions.
2. Add post-processing runner.
3. Add fulfillment adapter isolation.
4. Add fulfillment payload validation with ISO2 country normalization.
5. Add Django business-success inspection; HTTP 2xx alone must not complete fulfillment.
6. Persist `fulfillment_blocked` and `fulfillment_failed` states with request/response payloads.
7. Enqueue admin notification outbox rows for recoverable paid-order failures.
8. Trigger post-processing from webhook.

Step 5: Admin and recovery tooling

1. Add admin auth helper.
2. Add admin notification outbox query/send/resend helpers.
3. Add admin page + retry action.
4. Show fulfillment validation/provider failure details on the order recovery detail page.
5. Add dev fallback trigger endpoint.

Step 6: Cleanup

1. Remove runtime client post-processing code path from `usePayPalTXApproveCallback`.
2. Keep server actions (receipt/payment save/fulfillment) because webhook runner still uses them.

Rollback safety

- If webhook/post-processing breaks:
 - Keep ledger writes in place.
 - Disable webhook trigger path.
 - Use admin retry or dev fallback for recovery while patching.

27) Package Scripts (no extra Prisma config file)

Add scripts that explicitly target the ledger schema file.

```
JSON

{
  "scripts": {
    "prisma:merchize:generate": "prisma generate --config prisma.config.ts",
    "prisma:paypal:validate": "prisma validate --schema
prisma/shop/paypal/paypalTXLedger.schema.prisma",
    "prisma:paypal:format": "prisma format --schema prisma/shop/paypal/paypalTXLedger.schema.prisma",
    "prisma:paypal:generate": "prisma generate --schema
prisma/shop/paypal/paypalTXLedger.schema.prisma",
    "prisma:paypal:migrate:dev": "prisma migrate dev --schema
prisma/shop/paypal/paypalTXLedger.schema.prisma",
    "prisma:paypal:migrate:deploy": "prisma migrate deploy --schema
prisma/shop/paypal/paypalTXLedger.schema.prisma"
  }
}
```

Why this is safe:

- Merchize flow remains config-driven.
- PayPal ledger flow remains schema-path-driven.
- No cross-target confusion.

28) Testing Matrix (local/sandbox/live)

Local dev (no reliable webhook)

- Create intent from checkout.
- Authorize/capture routes write ledger payloads.

- Poll order status endpoint.
- Use dev fallback endpoint to trigger `runPostProcessing(orderToken)`.
- Verify:
 - receipt link/file saved
 - Django payment save custom ID saved
 - final status `completed`
 - invalid fulfillment payload becomes `fulfillment_blocked`, not a thrown-only error
 - Django `{ success: false }` with HTTP 200 becomes `fulfillment_failed`
 - fulfillment request/response payloads are saved for both blocked and failed cases
 - admin notification outbox row is created once per dedupe key

Sandbox webhook testing

- Enable webhook verification as needed.
- Confirm webhook rows dedupe by `eventId`.
- Confirm `completed` rows are not reprocessed.
- Confirm repeated webhook/post-processing attempts do not spam duplicate admin alerts.
- Confirm retry after `fulfillmentAddressOverride` can move a blocked/failed row to `completed`.

Production readiness checks

- Webhook verification ON.
- `ADMIN_SECRET_KEY` set.
- `ORDER_RECOVERY_ADMIN_EMAILS` set.
- Dev fallback route blocked in production.
- Monitoring/logs for:
 - `CREATE_ORDER_FAILED`
 - `AUTHORIZE_FAILED`
 - `CAPTURE_FAILED`
 - `FULFILLMENT_PAYLOAD_INVALID`
 - `FULFILLMENT_PROVIDER_REJECTED`
 - `POST_PROCESSING_FAILED`
- Outbox monitoring for:
 - pending admin notifications older than the expected send window
 - failed notification send attempts

29) Decision Defaults

These defaults are implemented in this guide's templates. Items marked **(deferred)** are intentionally kept simple for now with a known upgrade path.

1. `ORD_string` source

- Keep dual references.

- Internal primary key: `orderToken`
 - External/business fallback: `djangoOrderIntentOrderId ?? orderToken`
 - `djangoOrderIntentOrderId` comes from the verified Django order-intent OTP flow, not from a client-generated order string.
- ## 2. Webhook + sandbox testing mode
- Add dev-only fallback endpoint for `runPostProcessing(orderToken)`
 - Disabled in production
- ## 3. Admin auth transport **(deferred)**
- Header-only auth via `x-admin-secret`
 - No cookie/session logic in these templates
- ## 4. Status vocabulary **(deferred)**
- String statuses + centralized constants file
 - Prisma enum conversion deferred
- ## 5. Capture/authorize operation ownership
- Keep existing authorize/capture API routes as ownership point
 - Webhook is for confirmation, retries, and durable completion
 - Capture writes the immediate `captured` state; webhook + server runner own everything after that
- ## 6. Post-processing execution location
- Server-only (webhook + runner)
 - Client does polling + UI only
 - Client state is convenience only; ledger truth wins after refresh/reconnect
- ## 7. Error handling policy
- Persist `lastErrorCode` + `lastErrorMessage` on ledger row
 - Prefer structured route errors with stable `code`, `stage`, and `requestId`
- ## 8. Idempotency policy
- Track webhook delivery by `eventId`, but only mark it processed after business work succeeds
 - Use a DB-backed post-processing lease so only one worker owns receipt/payment/fulfillment at a time
 - Never rerun post-processing when ledger row already `completed`
- ## 9. Receipt timing
- Generate/upload receipt after `PAYMENT.CAPTURE.COMPLETED` only
- ## 10. Data retention baseline
- Keep raw authorize/capture payloads in JSON columns for reconciliation

30) Optional Overrides (only if rejecting defaults)

Use this section only if you intentionally want to diverge from Section 30.

1. `ORD_string` override

- Use `orderToken` only and stop storing the Django order-intent order ID.

2. Sandbox override

- Disable dev fallback endpoint and require webhook-only testing.

3. Admin auth override

- Replace header-only auth with cookie/session auth.

4. Status override

- Convert statuses to Prisma enum immediately.

5. Provider coupling override

- Inline Merchize calls directly in `runPostProcessing` (not recommended).

31) Future: Real-Time Notifications at Scale (SSE + Pub/Sub, not gRPC)

This section does not need to be implemented now. The polling approach in Section 19 is correct and sufficient. Read this when you are ready to replace polling with push-based real-time updates.

Decision default for this codebase:

- Polling first.
- SSE second when polling volume becomes wasteful.
- Keep the PayPal TX payment-status endpoint as the source of truth either way.
- Do not use gRPC for browser-facing payment-status delivery. If you ever introduce gRPC, reserve it for internal service-to-service boundaries, not confirmation-page client updates.

Why polling is fine for now

Polling the payment-status endpoint every 2–3 seconds works well when:

- You have fewer than ~500 concurrent users waiting on the confirmation page.
- Post-processing typically completes in under 30 seconds.
- Your VPS can absorb the request volume ($500 \text{ users} \times 1 \text{ req}/2.5\text{s} = 200 \text{ req/s}$ — manageable).

When polling stops being enough

At 5,000+ concurrent waiters, polling generates thousands of status requests per second. Most return "still processing" — wasted work. Server-Sent Events (SSE) invert this: the server pushes a single notification when the status actually changes.

Why NOT to use in-memory EventEmitter

A common first attempt at SSE:

TS

```
// DO NOT use this in production
import { EventEmitter } from 'events';
const emitter = new EventEmitter();

// Webhook writes to DB, then:
emitter.emit(`paypal-tx-payment-status:${orderToken}`, { status: 'completed' });

// SSE endpoint listens:
emitter.on(`paypal-tx-payment-status:${orderToken}`, (data) => {
  controller.enqueue(`data: ${JSON.stringify(data)}\n\n`);
});
```

This breaks because:

1. **Single-process only.** `EventEmitter` is in-memory. If you run PM2 cluster mode, multiple Node workers, or blue/green deploys, the webhook may hit worker A while the SSE connection is on worker B. Worker B never sees the event.
2. **No persistence.** If the user reconnects (browser tab refresh, network hiccup), the event is gone. The client must fall back to polling anyway.
3. **Memory leaks.** Each SSE connection registers a listener. If you forget to clean up on disconnect, you leak memory proportional to connection count.

What to use instead: Redis Pub/Sub or Postgres LISTEN/NOTIFY

Option A: Redis Pub/Sub (recommended for >1,000 concurrent waiters)

Redis Pub/Sub works across all Node processes and servers. Every worker subscribes; the webhook worker publishes.

TEXT

```
npm install ioredis
```

Publisher (inside `runPostProcessing` or `after()` callback, after status update):

TS

```
import Redis from 'ioredis';

const redisPub = new Redis(process.env.REDIS_URL!);

// After updating ledger status to 'completed':
await redisPub.publish(
  `paypal-tx-payment-status:${orderToken}`,
  JSON.stringify({ status: 'completed', completedAt: new Date().toISOString() }),
);
```

Subscriber (SSE endpoint):

```

import Redis from 'ioredis';

export async function GET(req: Request) {
  const url = new URL(req.url);
  const orderToken = url.searchParams.get('orderToken');
  if (!orderToken) return new Response('Missing orderToken', { status: 400 });

  const redisSub = new Redis(process.env.REDIS_URL!);
  const channel = `paypal-tx-payment-status:${orderToken}`;

  const stream = new ReadableStream({
    start(controller) {
      // Send heartbeat every 30s to keep the connection alive
      const heartbeat = setInterval(() => {
        controller.enqueue(`: heartbeat\n\n`);
      }, 30_000);

      redisSub.subscribe(channel);

      redisSub.on('message', (_ch, message) => {
        controller.enqueue(`data: ${message}\n\n`);
        // Clean up after delivering the terminal event
        clearInterval(heartbeat);
        redisSub.unsubscribe(channel);
        redisSub.quit();
        controller.close();
      });

      // Clean up if client disconnects
      req.signal.addEventListener('abort', () => {
        clearInterval(heartbeat);
        redisSub.unsubscribe(channel);
        redisSub.quit();
        controller.close();
      });
    },
  });

  return new Response(stream, {
    headers: {
      'Content-Type': 'text/event-stream',
      'Cache-Control': 'no-cache',
      Connection: 'keep-alive',
      'X-Accel-Buffering': 'no', // disable Nginx buffering
    },
  });
}

```

Option B: Postgres LISTEN/NOTIFY (no extra infrastructure)

If you do not want to add Redis, Postgres has built-in pub/sub via `LISTEN` / `NOTIFY`. This works across all Node processes connected to the same database.

Publisher (raw SQL after ledger status update):

TS

```
await prisma.$executeRawUnsafe(
  `NOTIFY order_status, '${JSON.stringify({ orderToken, status: 'completed' })}'`,
);
```

Subscriber (SSE endpoint, using a raw pg connection):

```

import { Client } from 'pg';

export async function GET(req: Request) {
  const url = new URL(req.url);
  const orderToken = url.searchParams.get('orderToken');
  if (!orderToken) return new Response('Missing orderToken', { status: 400 });

  const pgClient = new Client(process.env.DATABASE_URL!);
  await pgClient.connect();
  await pgClient.query('LISTEN order_status');

  const stream = new ReadableStream({
    start(controller) {
      const heartbeat = setInterval(() => {
        controller.enqueue(': heartbeat\n\n');
      }, 30_000);

      pgClient.on('notification', (msg) => {
        if (!msg.payload) return;
        const data = JSON.parse(msg.payload);
        if (data.orderToken !== orderToken) return;

        controller.enqueue(`data: ${msg.payload}\n\n`);
        clearInterval(heartbeat);
        pgClient.end();
        controller.close();
      });

      req.signal.addEventListener('abort', () => {
        clearInterval(heartbeat);
        pgClient.end();
        controller.close();
      });
    },
  });

  return new Response(stream, {
    headers: {
      'Content-Type': 'text/event-stream',
      'Cache-Control': 'no-cache',
      Connection: 'keep-alive',
      'X-Accel-Buffering': 'no',
    },
  });
}

```

Postgres LISTEN/NOTIFY limitations:

- Payload max 8,000 bytes (fine for status updates, not for large data).
- Each SSE connection holds a dedicated pg connection. At 1,000+ concurrent, you will hit your Postgres `max_connections` limit. Use Redis Pub/Sub instead at that scale.

- Notifications are fire-and-forget — if the listener connects after the NOTIFY, it misses the event. Always check current DB state on SSE connect (same as Redis).

Choosing between the two

Factor	Redis Pub/Sub	Postgres LISTEN/NOTIFY
Extra infrastructure	Yes (Redis server)	No (your existing Postgres)
Max concurrent SSE connections	10,000+ (limited by Node memory)	~200–500 (limited by pg connections)
Your VPS already runs Redis?	Use this	—
You want zero new dependencies	—	Use this

Keep polling as fallback

Regardless of which push mechanism you choose, **keep the payment-status polling endpoint**. SSE connections can drop due to proxies, network changes, or browser throttling (background tabs). The client should:

1. Open SSE connection on mount.
2. If SSE delivers the terminal status, done.
3. If SSE disconnects or times out, fall back to polling.
4. On reconnect, check current DB state first (don't wait for a new pub/sub event).

VPS proxy configuration (Nginx/Caddy)

Reverse proxies buffer responses by default, which breaks SSE. Add these headers:

Nginx:

NGINX

```
location /next-api/paypal/tx-ledger/payments/<orderToken>/status/stream {
    proxy_pass http://localhost:3000;
    proxy_http_version 1.1;
    proxy_set_header Connection '';
    proxy_buffering off;
    proxy_cache off;
    proxy_read_timeout 86400s; # keep SSE alive for up to 24h
}
```

Caddy (automatic, no config needed — Caddy streams by default when `X-Accel-Buffering: no` is set).